

3. Gradient Descent Methods

Optimization T2026

Master in Fundamental Principles of Data Science



UNIVERSITAT_{DE}
BARCELONA

$$f(\mathbf{x}) \rightarrow \min, \quad \mathbf{x} \in D \subseteq \mathbb{R}^n, \quad n \geq 1, \quad f \text{ is smooth}$$

Goal: Iteratively find a sequence $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots \rightarrow \mathbf{x}^*$,
where $\mathbf{x}^*$ is a solution of the optimization problem
(local or global minimum), realizing the descent

$$f(\mathbf{x}^{(1)}) > f(\mathbf{x}^{(2)}) > \dots$$

(for all or most* of the iterates)

Recall that
 $\nabla f(\mathbf{x}^*) = 0$

General descent method.

given a starting point $\mathbf{x}^{(1)} \in D$

repeat

1. Determine *descent direction* $\mathbf{p}^{(k)}$ (often, $\|\mathbf{p}^{(k)}\| = 1$)
2. Determine *step size/learning rate* $\alpha^{(k)}$
3. Update $\mathbf{x}^{(k+1)} := \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)}$

until *stopping criterion* is satisfied

III. Descent direction?

II. Step size?

I. Stopping criterion?

Digression: Why gradient?

Recall that from the Taylor formula

$$\begin{aligned} f(\mathbf{x} + \mathbf{v}) &= \text{up to H.O.T. } f(\mathbf{x}) + \mathbf{v}^T \nabla f(\mathbf{x}) \\ &= f(\mathbf{x}) + \nabla_{\mathbf{v}} f(\mathbf{x}) \end{aligned}$$

Directional
derivative

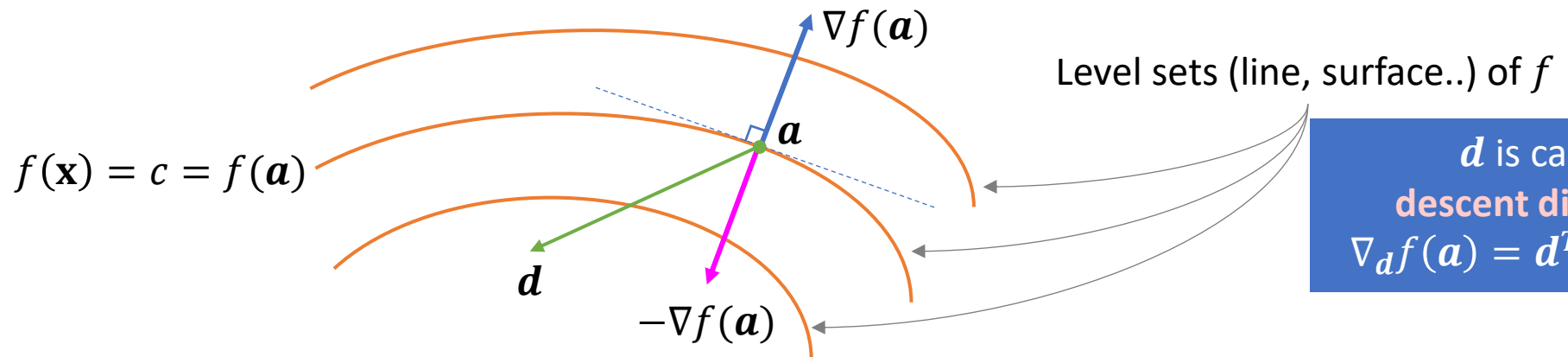
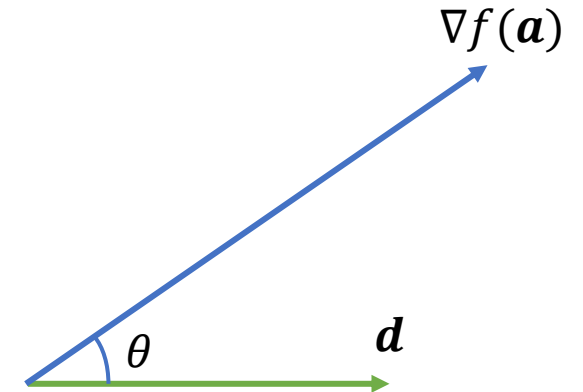
We want it to
be negative

Theorem:

Let $f: D \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ be a differentiable function, $\mathbf{a} \in D$, $\mathbf{d} \in \mathbb{R}^n$ with $\|\mathbf{d}\| = 1$. If θ is the angle between $\mathbf{d}$ and $\nabla f(\mathbf{a})$. Then

$$\nabla_{\mathbf{d}} f(\mathbf{a}) = \mathbf{d}^T \nabla f(\mathbf{a}) = \|\nabla f(\mathbf{a})\| \cos \theta$$

In particular, the vector $-\nabla f(\mathbf{a})$ gives the maximum descent direction of f at the point $\mathbf{a}$ (provided $\nabla f(\mathbf{a}) \neq 0$).



$\mathbf{d}$ is called a
descent direction if
 $\nabla_{\mathbf{d}} f(\mathbf{a}) = \mathbf{d}^T \nabla f(\mathbf{a}) < 0$

I. Stopping criteria/termination conditions

- **Maximum iterations:** repeat until $k \leq k_{max}$

- **Absolute improvement:** repeat until

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_a$$

- **Relative improvement:** repeat until

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_r |f(\mathbf{x}^{(k)})|$$

- **Gradient magnitude:** repeat until

$$\|\nabla f(\mathbf{x}^{(k+1)})\| < \epsilon_g$$

-
- ✓ One or more termination conditions can be used
 - ✓ If there are several local minima, one can add **random restart** with $\mathbf{x}^{(1),new}$ sampled randomly from D

II. Step size/learning rate

Suppose $x = x^{(k)}$ and $p = p^{(k)}$ is given. How to find $\alpha = \alpha^{(k)}$?

Methods:

1. Exact line search
2. Approximate line search
3. Trust region methods

Exact line search

$$\text{minimize}_{\alpha} f(\mathbf{x} + \alpha \mathbf{p})$$

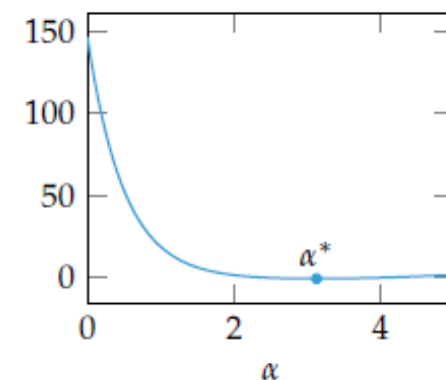
Consider conducting a line search on $f(x_1, x_2, x_3) = \sin(x_1 x_2) + \exp(x_2 + x_3) - x_3$ from $\mathbf{x} = [1, 2, 3]$ in the direction $\mathbf{d} = [0, -1, -1]$. The corresponding optimization problem is:

$$\text{minimize}_{\alpha} \sin((1 + 0\alpha)(2 - \alpha)) + \exp((2 - \alpha) + (3 - \alpha)) - (3 - \alpha)$$

which simplifies to:

$$\text{minimize}_{\alpha} \sin(2 - \alpha) + \exp(5 - 2\alpha) + \alpha - 3$$

The minimum is at $\alpha \approx 3.127$ with $\mathbf{x} \approx [1, -1.126, -0.126]$.



Exact line search

$$\text{minimize}_{\alpha} f(\mathbf{x} + \alpha \mathbf{p})$$

- This is a univariate optimization problem for $\phi(\alpha) := f(\mathbf{x} + \alpha \mathbf{p}) \rightarrow$

→ Find a **bracket** for the optimal solution α^*

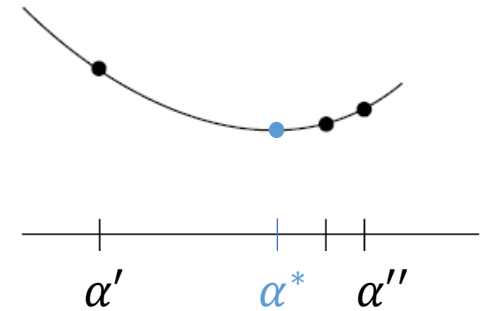
(α^* is characterized by $\phi(\alpha^*) < \phi(\alpha)$ for all α near α^*)

→ Use univariate optimization methods to find an approximation of α^* by successively shrinking the bracket. Methods include:

- Dyadic/binary search
- Fibonacci search
- Quadratic fit search
- Shubert–Piyavskii method
- Bisection method

} Only for unimodal functions!

Definition: A *bracket* is an interval $[\alpha', \alpha'']$ containing α^*



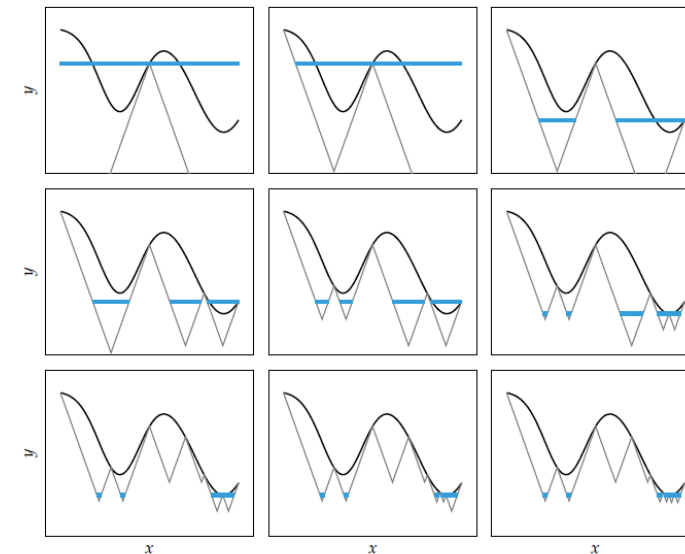
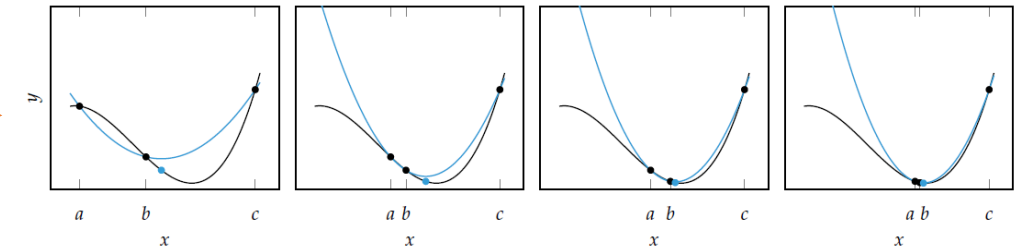
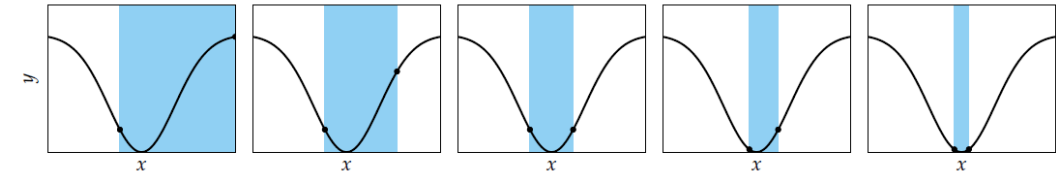
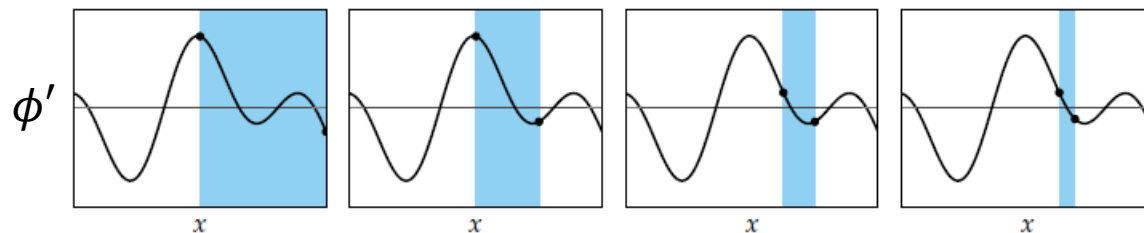
Digression: some univalent optimization methods

[Kochenderfer – Wheeler, Ch.3]

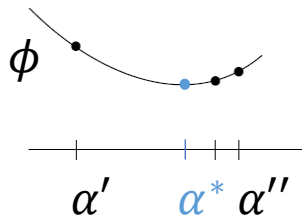
- **Dyadic/binary search**: subdivide interval 'in half' at each step
- **Fibonacci search**: max reduction of interval size for given number of function evaluations
- **Quadratic fit search**
- **Shubert-Piyavskii method**: assuming ϕ is Lipschitz, e.g.

$$|\phi(x) - \phi(y)| \leq \ell \cdot |x - y|, \quad \forall x, y \in [\alpha', \alpha'']$$

- **Bisection method**: solve $\phi'(\alpha) = 0$ instead



Dyadic/binary and Fibonacci search



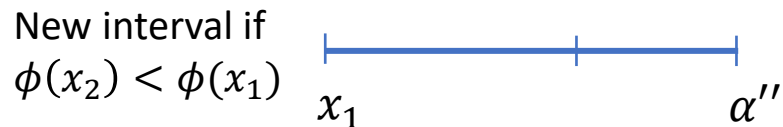
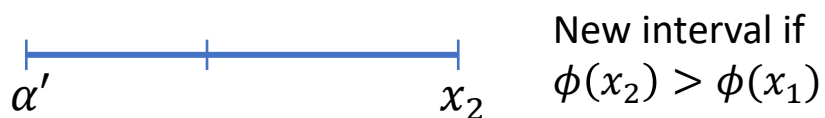
Assumption ★:

ϕ is **unimodal**, that is, ϕ has a unique minimum on (α', α'')

ϕ is decreasing on $[\alpha', \alpha^*]$ and ϕ is increasing on $[\alpha^*, \alpha'']$

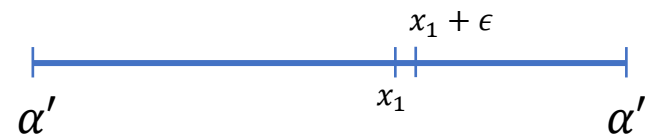
ϕ is convex on $[\alpha', \alpha'']$
($\Leftrightarrow \phi'' > 0$)

Basic Splitting Step: for a pair $x_1 < x_2$ of points in the starting bracket



Exercise: Check that, under Assumption ★, for all $x_1 < x_2$ after the Basic Splitting Step the new interval contains α^* (hence, is a bracket).

Basic Splitting Step in “almost” two parts:
do the basic splitting step for x_1 and $x_1 + \epsilon$, where $\epsilon > 0$ is small



- Each Basic Splitting Step requires 2 evaluations of the function at x_1 and x_2 .
- In general, i.e., if Assumption ★ is violated, the Basic Splitting Step doesn't work!

Exercise: Give an example

Dyadic/binary search:
under **Assumption** ★

given the desired size $\epsilon > 0$ of the bracket

choose $\delta < \epsilon$ (usually much smaller)

repeat

1. Pick the midpoint $x_1 = \frac{\alpha' + \alpha''}{2}$

2. Do the **Basic Splitting Step in 'almost' two parts** using x_1 and $x_1 + \delta$

3. Update $[\alpha', \alpha'']$ with the new bracket from step 2 above

until $|\alpha'' - \alpha'| < \epsilon$

Exercise: How many evaluations of the function ϕ is required in the dyadic search in order to shrink the bracket by a factor of 100?

Fibonacci search (under Assumption ★)

- Fibonacci numbers are given by the recursive relation $F_{n+2} = F_{n+1} + F_n$, with starting condition $F_1 = F_2 = 1$.
- This generates the sequence 1, 1, 2, 3, 5, 8, 11, ...
- This sequence grows as $F_n \sim \frac{1}{\sqrt{5}} \varphi^n$, for n large, where $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.6$ is the Golden ratio.

given the number of steps N

for $i = N, N - 1, \dots, 1$ **do**

if $i \neq 1$,

 1. Compute $x_1, x_2 \in [\alpha', \alpha'']$ such that

$$\frac{\alpha'' - x_1}{\alpha'' - \alpha'} = \frac{F_i}{F_{i+1}} \text{ and } \frac{x_2 - \alpha'}{\alpha'' - \alpha'} = \frac{F_i}{F_{i+1}}$$

 2. Do the **Basic Splitting Step** using x_1 and x_2

 3. Update $[\alpha', \alpha'']$ with the new bracket from step 2 above

otherwise

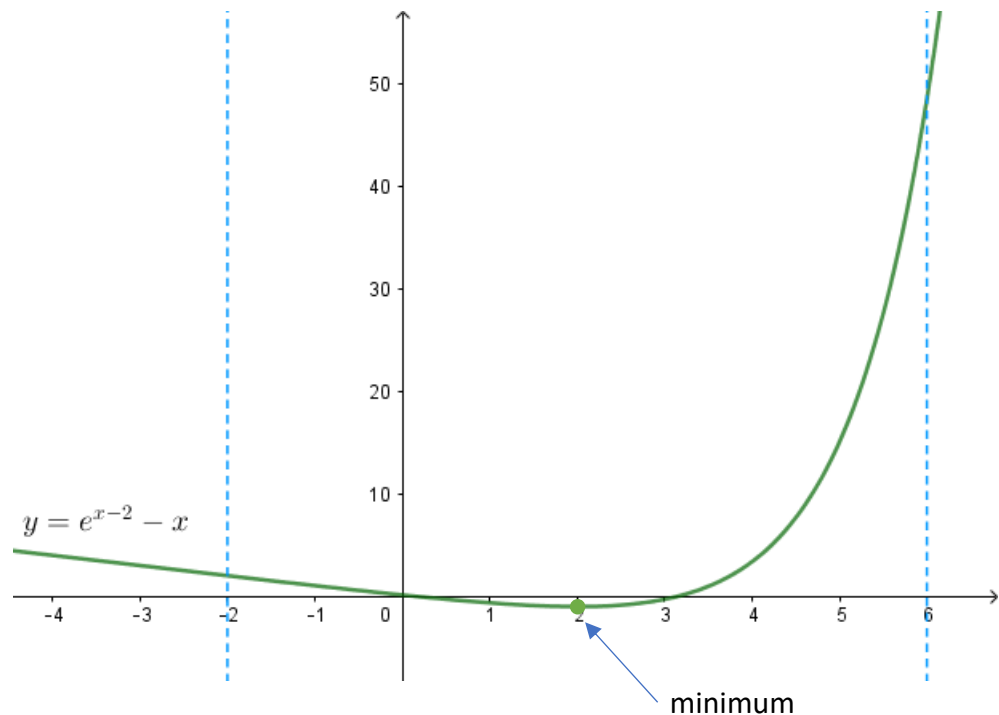
 Do the **Basic Splitting Step in 'almost' two parts** using $\frac{\alpha' + \alpha''}{2}$ and $\frac{\alpha' + \alpha''}{2} + \epsilon$

Observe that after this step, the length of the new bracket is proportional to the length of the previous bracket as F_i to F_{i+1}

Key Advantage: Fibonacci search uses significantly smaller evaluations of the function than the dyadic search because it re-uses some evaluation points! (see example on the next slide)

Exercise: How many evaluations of the function ϕ is required in the Fibonacci search in order to shrink the bracket by a factor of 100? Compare it to the corresponding result of the dyadic search.

Fibonacci search (under Assumption ★): an example



Consider using Fibonacci search with five function evaluations to minimize $f(x) = \exp(x - 2) - x$ over the interval $[a, b] = [-2, 6]$. The first two function evaluations are made at $\frac{F_5}{F_6}$ and $1 - \frac{F_5}{F_6}$, along the length of the initial bracketing interval:

$$f(x^{(1)}) = f\left(a + (b - a)\left(1 - \frac{F_5}{F_6}\right)\right) = f(1) = -0.632$$

$$f(x^{(2)}) = f\left(a + (b - a)\frac{F_5}{F_6}\right) = f(3) = -0.282$$

The evaluation at $x^{(1)}$ is lower, yielding the new interval $[a, b] = [-2, 3]$. Two evaluations are needed for the next interval split:

$$x_{\text{left}} = a + (b - a)\left(1 - \frac{F_4}{F_5}\right) = 0$$

$$x_{\text{right}} = a + (b - a)\frac{F_4}{F_5} = 1$$

A third function evaluation is thus made at x_{left} , as x_{right} has already been evaluated:

$$f(x^{(3)}) = f(0) = 0.135$$

The evaluation at $x^{(1)}$ is lower, yielding the new interval $[a, b] = [0, 3]$. Two evaluations are needed for the next interval split:

$$x_{\text{left}} = a + (b - a)\left(1 - \frac{F_3}{F_4}\right) = 1$$

$$x_{\text{right}} = a + (b - a)\frac{F_3}{F_4} = 2$$

A fourth functional evaluation is thus made at x_{right} , as x_{left} has already been evaluated:

$$f(x^{(4)}) = f(2) = -1$$

The new interval is $[a, b] = [1, 3]$. A final evaluation is made just next to the center of the interval at $2 + \epsilon$, and it is found to have a slightly higher value than $f(2)$. The final interval is $[1, 2 + \epsilon]$.

Quadratic fit search

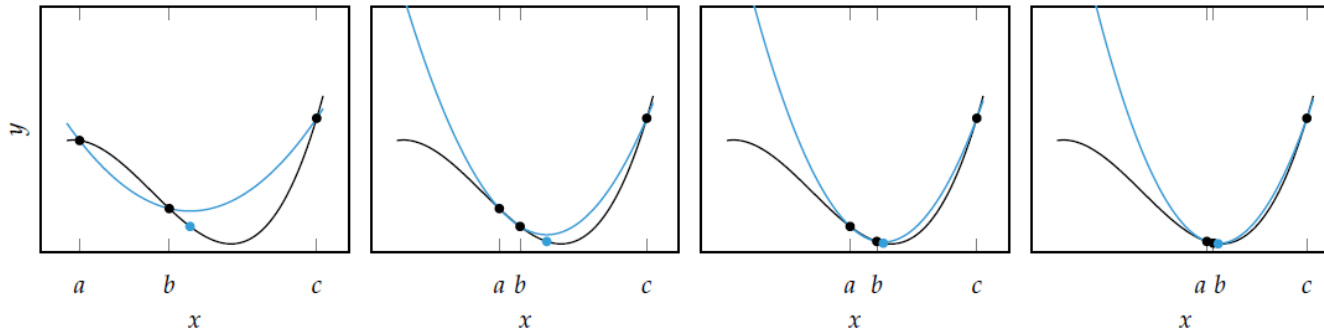
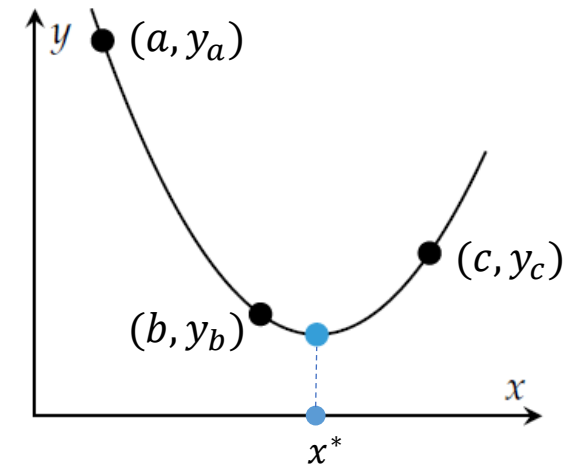
The method is based on the following observations:

- 'close' to the minima functions look like quadratic functions
- we can explicitly find minima of quadratic functions:

Lemma:

There exists a unique parabola that passes through any triple of distinct points (a, y_a) , (b, y_b) , (c, y_c) . This parabola has its extremum at

$$x^* = \frac{1}{2} \frac{y_a(b^2 - c^2) + y_b(c^2 - a^2) + y_c(a^2 - b^2)}{y_a(b - c) + y_b(c - a) + y_c(a - b)}$$



Exercise*: Show that the algorithm described on the next slide converges to a local minimum (assuming the function is smooth)

Quadratic fit search

given a triple $a < b < c$ where $[a, c]$ is a bracket of ϕ and $\phi(b) < \phi(a)$, $\phi(b) < \phi(c)$

repeat

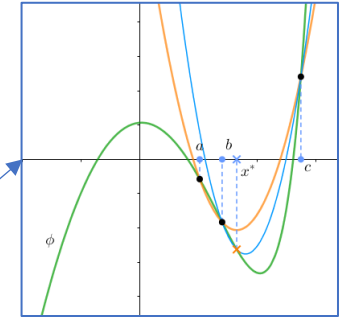
1. Compute the critical point x^* of the parabola passing through a, b, c
2. If $x^* \in [b, c]$, then
 - Check which value is larger, $\phi(x^*)$ or $\phi(b)$:
 - i. If $\phi(b) > \phi(x^*)$, update the triple (a, b, c) with (b, x^*, c)
 - ii. If $\phi(b) < \phi(x^*)$, update the triple (a, b, c) with (a, b, x^*)
3. Otherwise
 - Again, check which value is larger, $\phi(x^*)$ or $\phi(b)$:
 - i. If $\phi(b) > \phi(x^*)$, update the triple (a, b, c) with (a, x^*, b)
 - ii. If $\phi(b) < \phi(x^*)$, update the triple (a, b, c) with (x^*, b, c)

until the $|a - c| < \epsilon$

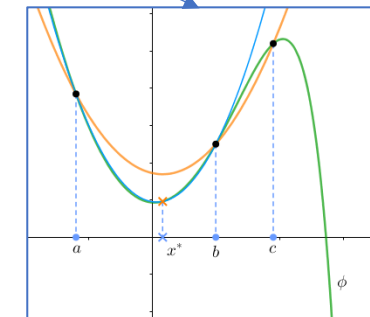
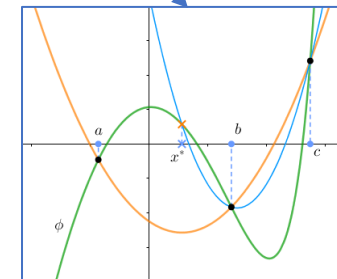
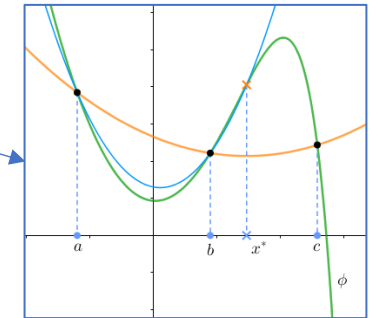
That is, when $x^* \in [a, b)$
because of condition #

Or any other stopping criterion
based on variation of the function

#



In these examples, parabola at the current step is in orange; parabola at the next step is in blue



Bisection method

The method is based on the following observations:

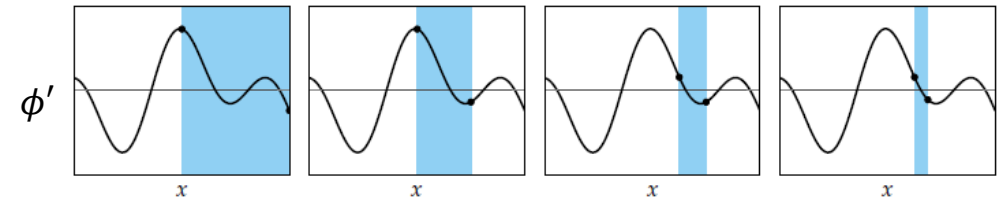
- Instead of looking for a local minimum of ϕ , we can look for a solution of $\phi' = 0$
- We assume that $[\alpha', \alpha'']$ is a bracket for ϕ , and hence there exists a solution of $\phi' = 0$ on this interval

given an interval $[a, b]$ such that $\phi'(a) \cdot \phi'(b) < 0$

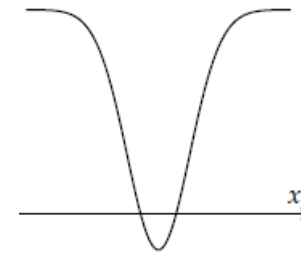
repeat

1. Compute the midpoint $c = \frac{a+b}{2}$
2. If $\phi'(a) \cdot \phi'(c) < 0$, update interval $[a, b]$ with $[a, c]$
3. If $\phi'(b) \cdot \phi'(c) < 0$, update interval $[a, b]$ with $[c, b]$

until $|a - b| < \epsilon$



- If $[\alpha', \alpha'']$ doesn't satisfy the condition $\phi'(\alpha') \cdot \phi'(\alpha'') < 0$, then one can try iteratively shrink this interval by a constant factor (say 2), until the condition is fulfilled. However, it might not always work (see an example of the function on the right where the bisection method can fail; this is the situation of a local minimum in a 'deep valley'). More sophisticated methods should be used instead.



Exercise: Let $\phi(x) = \frac{x^2}{2} - x$. Apply the bisection method to find an interval containing the minimizer of ϕ starting with the interval $[0, 1000]$. Execute 3 steps of the algorithm.

End of digression

Approximate line search

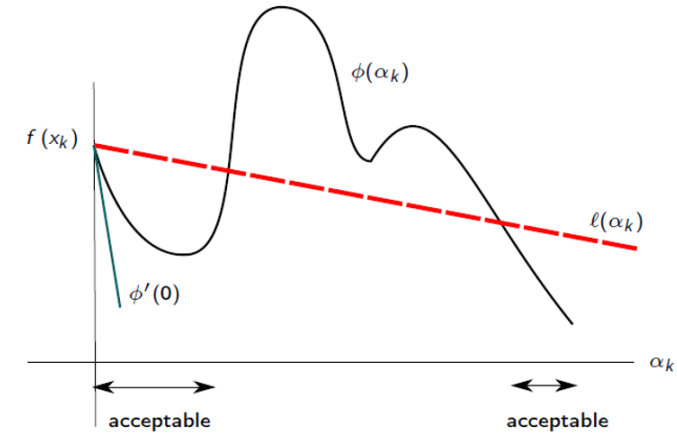
Find $\alpha^{(k)}$ so that the value $f(\mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{p}^{(k)})$ decreases (not necessarily best possible) and move on with the descent method

For simplicity, $x_k = \mathbf{x}^{(k)}, p_k = \mathbf{p}^{(k)}, \alpha_k = \alpha^{(k)}$

We impose the following condition for α_k

$$\phi(\alpha_k) := f(x_k + \alpha_k p_k) < f(x_k) + c_1 \alpha_k (\nabla f(x_k))^T p_k, \quad c_1 \in (0, 1).$$

The condition is called **(sufficient decrease condition)**.



Remarks.

- $\ell(\alpha_k) := f(x_k) + c_1 \alpha_k \nabla f^T(x_k) p_k$ is a linear function.
- For small values of $\alpha_k > 0$ we have $\phi(\alpha_k) < \ell(\alpha_k)$. This is so because $c_1 \in (0, 1)$ and then

$$\phi'(0) = (\nabla f(x_k))^T p_k < c_1 (\nabla f(x_k))^T p_k = \ell'(0) < 0.$$

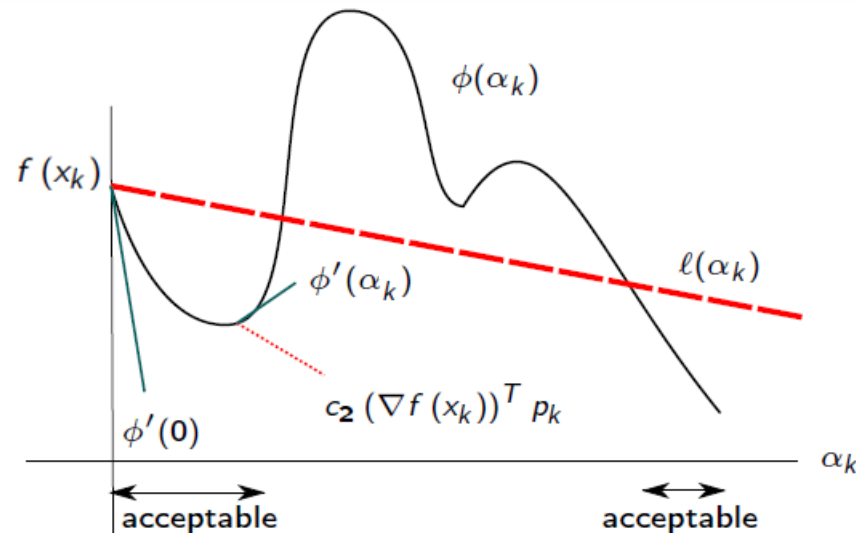
Recall: since p_k is a descent direction, we have $(\nabla f(x_k))^T p_k < 0$

Approximate line search

Curvature condition. Since the previous condition is always satisfied for small values of α_k we need to add further conditions for termination. We use the so called **curvature condition**

$$(\nabla f(x_k + \alpha_k p_k))^T p_k \geq c_2 (\nabla f(x_k))^T p_k, \quad c_2 \in (c_1, 1)$$

In other words if $\phi'(\alpha_k)$ is not **negative enough** we terminate the k -step.



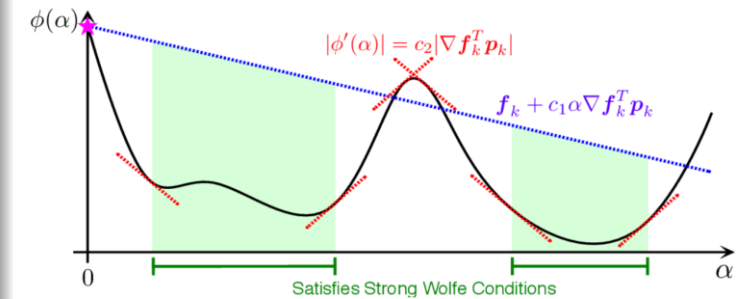
Approximate line search

Wolfe conditions

Definition. The conditions (together) to terminate the k -step given by

$$\begin{aligned} f(x_k + \alpha_k p_k) &< f(x_k) + c_1 \alpha_k (\nabla f(x_k))^T p_k, \\ (\nabla f(x_k + \alpha_k p_k))^T p_k &\geq c_2 (\nabla f(x_k))^T p_k, \end{aligned}$$

with $0 < c_1 < c_2 < 1$ are usually called **Wolfe conditions**.



Definition. The conditions (together) to terminate the k -step given by

$$\begin{aligned} f(x_k + \alpha_k p_k) &< f(x_k) + c_1 \alpha_k (\nabla f(x_k))^T p_k, \\ |(\nabla f(x_k + \alpha_k p_k))^T p_k| &\leq |c_2 (\nabla f(x_k))^T p_k|, \end{aligned}$$

with $0 < c_1 < c_2 < 1$ are usually called **strong Wolfe conditions**.

We do not allow $\phi'(\alpha_k)$ to be too positive

Approximate line search

Wolfe conditions: existence

Lemma. Suppose $f : D \subset \mathbb{R}^n \rightarrow \mathbb{R}$ be a $\mathcal{C}^1$ function. Let p_k a descent direction at the point $x_k \in D$ and assume $f|_{L_{p_k}}$ is bounded below where $L_{p_k} = \{x \in \mathbb{R}^n \mid x = x_k + \alpha p_k, \alpha > 0\}$. Then if $0 < c_1 < c_2 < 1$ there exist intervals of step lengths satisfying the (strong) Wolfe conditions

Proof. Since $\ell'(\alpha_k) < 0$ (and constant) there exists a first intersection, $\hat{\alpha}_k > 0$, between $\ell(\alpha_k)$ and $\phi(\alpha_k)$:

$$f(x_k + \hat{\alpha}_k p_k) = f(x_k) + c_1 \hat{\alpha}_k (\nabla f(x_k))^T p_k.$$

The sufficient decrease condition is satisfied for all $\alpha_k \in [0, \hat{\alpha}_k]$. By the Mean Value Theorem we have that there exists $\tilde{\alpha}_k \in [0, \hat{\alpha}_k]$ such that

$$f(x_k + \hat{\alpha}_k p_k) - f(x_k) = \hat{\alpha}_k (\nabla f(x_k + \tilde{\alpha}_k p_k))^T p_k$$

All together imply

$$(\nabla f(x_k + \tilde{\alpha}_k p_k))^T p_k = c_1 \hat{\alpha}_k (\nabla f(x_k))^T p_k > c_2 \hat{\alpha}_k (\nabla f(x_k))^T p_k.$$

Therefore $\tilde{\alpha}_k$ satisfies the Wolfe conditions and smoothness gives the desired interval.

Approximate line search

Convergence

Remark. Until this moment we just consider the **definition of the process**, that is the election of p_k and α_k . But we need to study if the process converge to **somewhere**.

Let p_k be a descent direction, and let θ_k the angle of p_k and $-\nabla f(x_k)$:

$$\cos(\theta_k) = -\frac{1}{\|\nabla f(x_k)\| \|p_k\|} (\nabla f(x_k))^T p_k$$

Theorem. Assume notation above with p_k a descent direction and α_k satisfying Wolfe's conditions. Suppose f is $\mathcal{C}^2$ and bounded below in $\mathbb{R}^n$. Then

$$\sum_{k=0}^{\infty} \cos^2(\theta_k) \|\nabla f(x_k)\| < \infty.$$

Approximate line search

Convergence

Corollary. Under the above notation and assumptions we have

$$\cos^2(\theta_k) \|\nabla f(x_k)\| \rightarrow 0$$

Moreover if there exists $\delta > 0$ such that $\cos(\theta) > \delta$ then

$$\lim_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0 \quad (\text{globally convergent algorithms})$$

Remark. The final δ -condition basically means that p_k do not get arbitrarily **orthogonal** to the gradient vector. This is, for instance, the case of the **steepest descent method**.

Approximate line search

Convergence

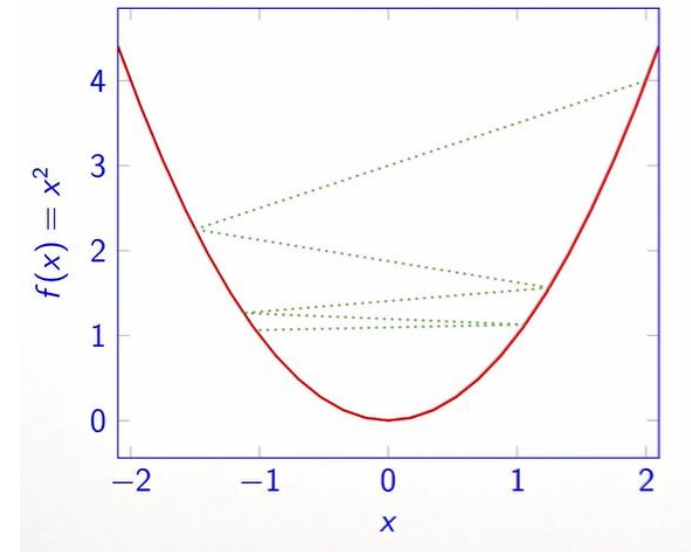
Problem: Consider the function $f(x) = x^2$ on $[-2, 2]$. Consider the one-dimensional gradient descent method starting at $\mathbf{x}_0 = 2$ in the direction

$$\mathbf{p}_k = -\text{sign}(\mathbf{x}_k)$$

with step

$$\alpha_k = 2 + 3(2^{-k-1}).$$

- 1) Verify that $\mathbf{p}_k$ is indeed a descent direction, that is, $f(\mathbf{x}_{k+1}) < f(\mathbf{x}_k)$.
- 2) Perform 5 steps of the descent algorithm.
- 3) Does this descent converge? (*Hint: see picture on the right.*) Justify your argument. What Wolfe conditions are violated?

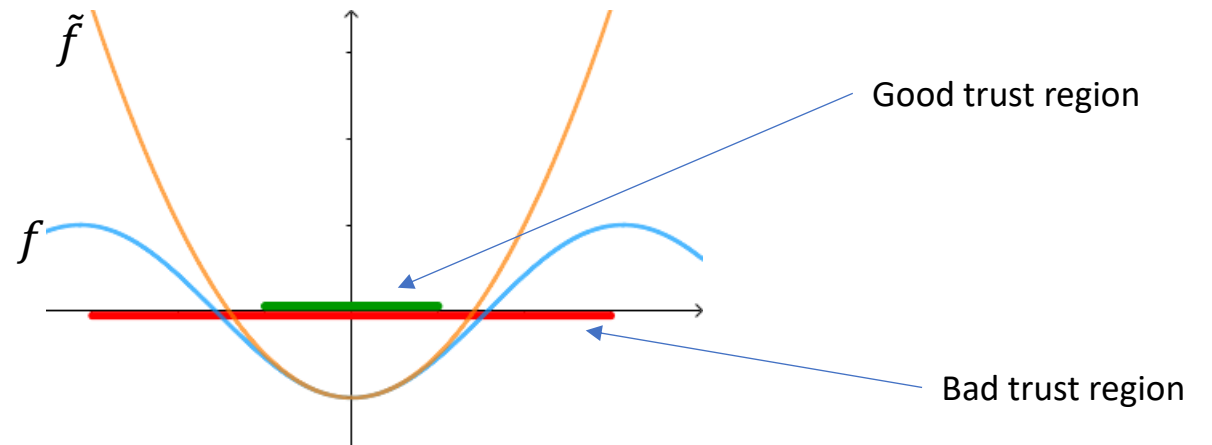


Trust region method

Idea

- Line search methods: find a descent direction $\rightarrow$ find the next point in this direction
- Trust region methods: find a region 'of possible good steps' $\rightarrow$ find a point in this region

Usually, we approximate the objective function f with a simpler objective $\tilde{f}$.



Potential problem: It might be that the solution $\tilde{x}^*$ of $\min \tilde{f}(x)$ lies in the region where $\tilde{f}$ badly approximates f

A solution: restrict the optimization of $\tilde{f}$ to the region where we **trust** that $\tilde{f}$ is a good approximation of f

Trust region method

Idea (cont.)

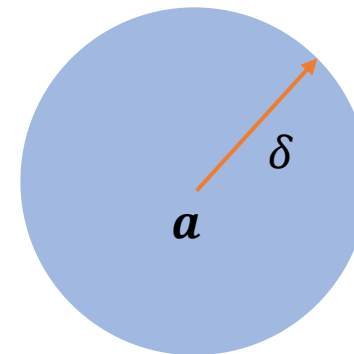
Typically, near a point $\mathbf{a}$ we do the quadratic approximation

$$f(\mathbf{x}) \approx \tilde{f}(\mathbf{x}) = f(\mathbf{a}) + \nabla f(\mathbf{a})^T \cdot (\mathbf{x} - \mathbf{a}) + \frac{1}{2} (\mathbf{x} - \mathbf{a})^T \cdot \nabla^2 f(\mathbf{a}) \cdot (\mathbf{x} - \mathbf{a})$$

At $\mathbf{a}$, f and $\tilde{f}$ match: $f(\mathbf{a}) = \tilde{f}(\mathbf{a})$
The further we go from $\mathbf{a}$, the worse the approximation is

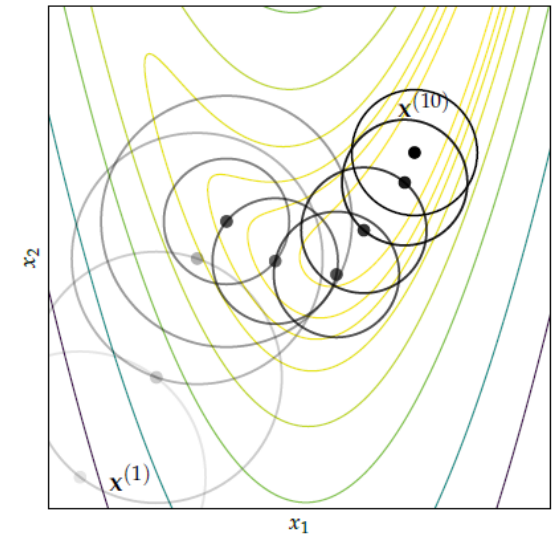
A trust region is often chosen to be a ball of radius $\delta > 0$ centered at $\mathbf{a}$:

$$\{\mathbf{x} \in \mathbb{R}^n \mid \|\mathbf{x} - \mathbf{a}\| \leq \delta\}$$



Trust region method

Generic algorithm



given $\delta, \mathbf{x}_1$ and $k = 0$

repeat

1. $k \leftarrow k + 1$
2. Find a solution $\mathbf{x}_k^*$ of the minimization problem $\tilde{f} \rightarrow \min$ subject to $\|\mathbf{x} - \mathbf{x}_{k-1}^*\| \leq \delta$
3. If $\tilde{f}(\mathbf{x}_k^*) \approx f(\mathbf{x}_k^*)$, then increase δ
else decrease δ

Trust region subproblem

For example, for

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}_{k-1}^*) + \nabla f(\mathbf{x}_{k-1}^*)^T \cdot (\mathbf{x} - \mathbf{x}_{k-1}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_{k-1}^*)^T \cdot \nabla^2 f(\mathbf{x}_{k-1}^*) \cdot (\mathbf{x} - \mathbf{x}_{k-1}^*)$$

until the required precision is reached

For example: compute the *predictive performance*

$$\eta = \frac{\text{actual improvement}}{\text{predicted improvement}} = \frac{|f(\mathbf{x}_{k-1}^*) - f(\mathbf{x}_k^*)|}{|f(\mathbf{x}_{k-1}^*) - \tilde{f}(\mathbf{x}_k^*)|} \in (0,1]$$

- If $\eta < \eta_1$, then $\delta \leftarrow \delta / \gamma_1$, for $\gamma_1 > 1$
- If $\eta > \eta_2$, then $\delta \leftarrow \delta \cdot \gamma_2$, for $\gamma_2 > 1$

$= \tilde{f}(\mathbf{x}_{k-1}^*)$

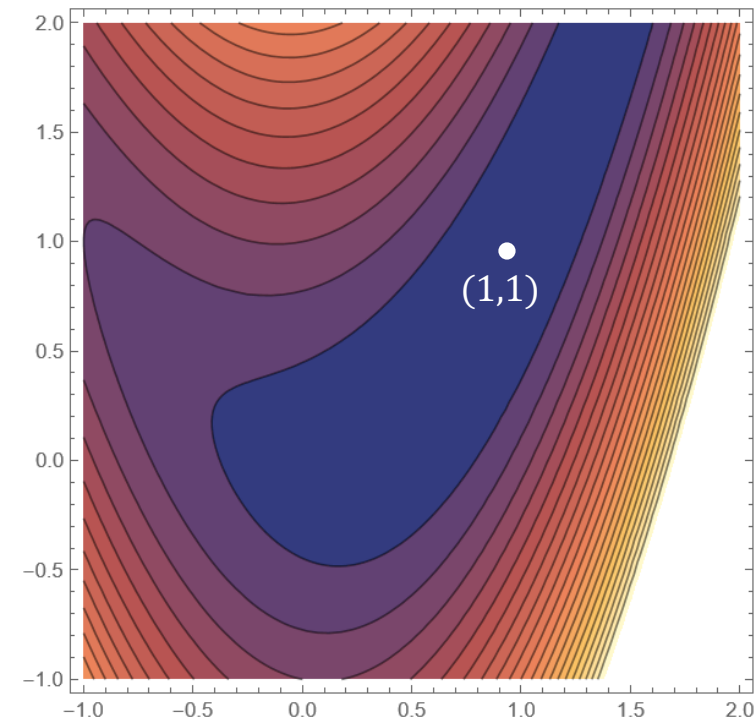
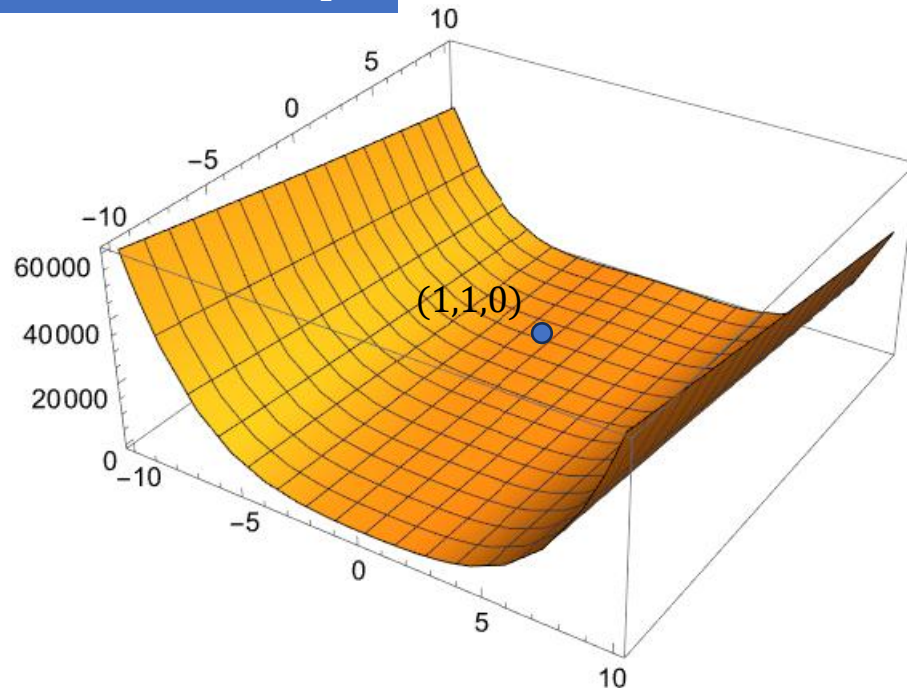
Trust region method

Example

The **Rosenbrock function** $f(x) = (a - x_1)^2 + b(x_2 - x_1^2)^2$

Global minimum at $\mathbf{x}^* = (a, a^2)$ **Exercise:** Check

$$a = 1, b = 5$$
$$f(x) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$$

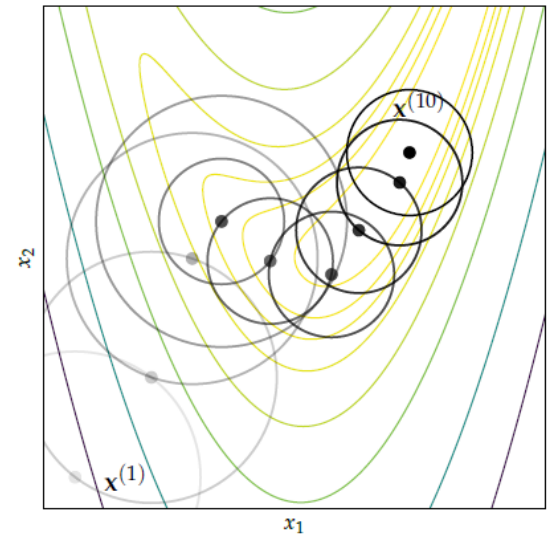
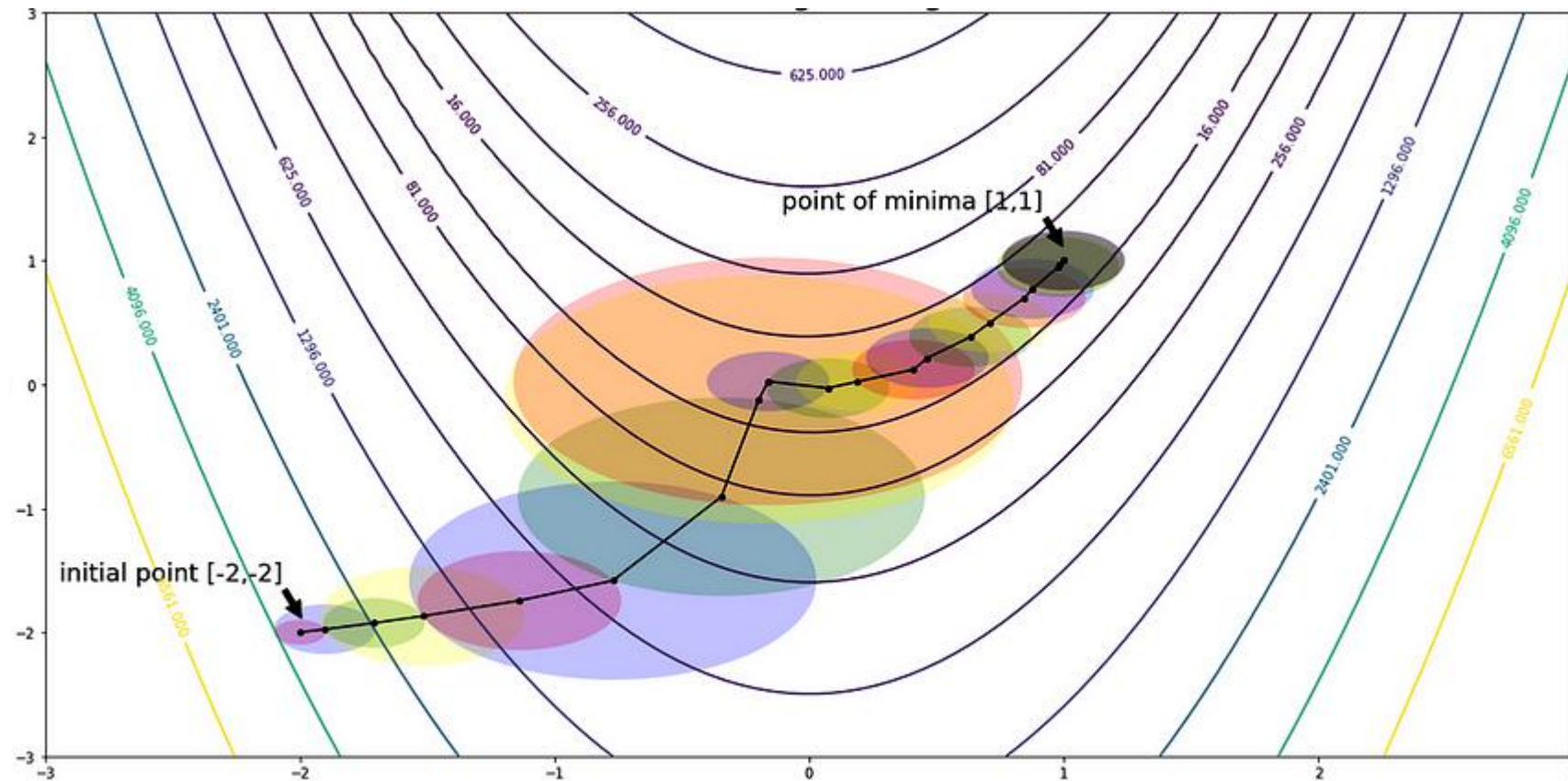


Trust region method

Example

The **Rosenbrock function** $f(x) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$

Global minimum at $\mathbf{x}^* = (1,1)$



$$a = 1, b = 5$$

Trust region subproblem

How to solve?

Fix k and assume $\mathbf{x}_{k-1}^*$ is given. Let us re-write

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}_{k-1}^*) + \nabla f(\mathbf{x}_{k-1}^*)^T \cdot (\mathbf{x} - \mathbf{x}_{k-1}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_{k-1}^*)^T \cdot \nabla^2 f(\mathbf{x}_{k-1}^*) \cdot (\mathbf{x} - \mathbf{x}_{k-1}^*)$$

using

$$G_k := \nabla f(\mathbf{x}_{k-1}^*)^T \text{ (gradient),}$$

$$H_k := \nabla^2 f(\mathbf{x}_{k-1}^*) \text{ (Hessian),}$$

$$\mathbf{p}_k = \mathbf{x} - \mathbf{x}_{k-1}^* \text{ (step),}$$

$$f_k = f(\mathbf{x}_{k-1}^*)$$

as

$$m_k(\mathbf{p}_k) = \tilde{f}(\mathbf{p}_k + \mathbf{x}_{k-1}^*) = f_k + G_k \cdot \mathbf{p}_k + \frac{1}{2} \mathbf{p}_k^T \cdot H_k \cdot \mathbf{p}_k.$$

We need to solve

$$m_k(\mathbf{p}_k) \rightarrow \min, \text{ subject to } \|\mathbf{p}_k\| \leq \delta$$

Trust region subproblem

How to solve? (cont.) Cauchy point

$$m_k(\mathbf{p}_k) \rightarrow \min, \text{ subject to } \|\mathbf{p}_k\| \leq \delta$$

Define a **Cauchy point** $\mathbf{p}_k^C$ via the following steps:

1. Find the point $\mathbf{p}_k^\ell$ that minimizes the **linear** part of m_k :

$$\mathbf{p}_k^\ell = \arg \min_{\mathbf{p} \in \mathbb{R}^n} (f_k + G_k \cdot \mathbf{p}), \quad \|\mathbf{p}\| \leq \delta$$

Since $f_k + G_k \cdot \mathbf{p} \geq f_k - \|G_k\| \cdot \|\mathbf{p}\| \geq f_k - \delta \cdot \|G_k\|$, we obtain that

$$\mathbf{p}_k^\ell = -\delta \frac{G_k}{\|G_k\|}$$

The point $\mathbf{p}_k^\ell$ is a 'poor' approximation, so:

2. Compute the scalar $\tau_k > 0$ that minimizes $m_k(\tau_k \mathbf{p}_k^\ell)$ subject to the trust region bound:

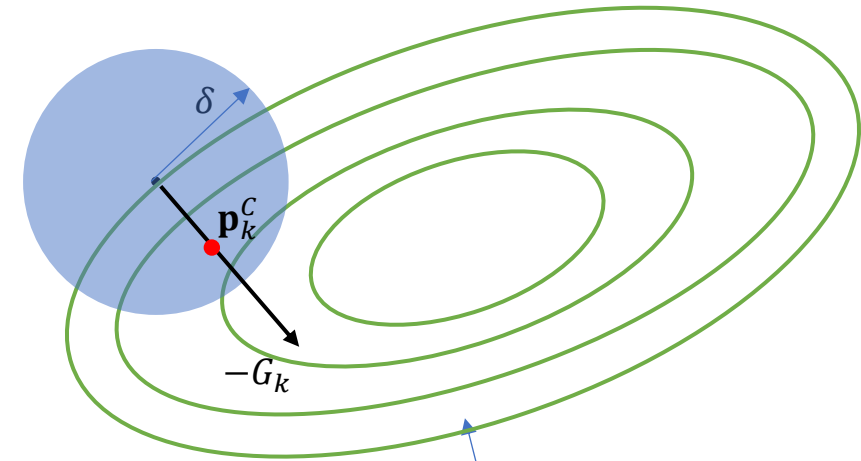
$$\tau_k = \arg \min_{\tau \in \mathbb{R}} m_k(\tau \mathbf{p}_k^\ell), \quad \|\tau \mathbf{p}_k^\ell\| \leq \delta$$

Solving quadratic inequality, we get that $\tau_k = \begin{cases} 1, & \text{if } G_k^T \cdot H_k \cdot G_k \leq 0 \\ \min\{1, \hat{\tau}_k\}, & \text{otherwise} \end{cases}$

$$\text{where } \hat{\tau}_k = \frac{\|G_k\|^3}{\delta G_k^T \cdot H_k \cdot G_k}.$$

3. Set $\mathbf{p}_k^C = \tau_k \mathbf{p}_k^\ell$

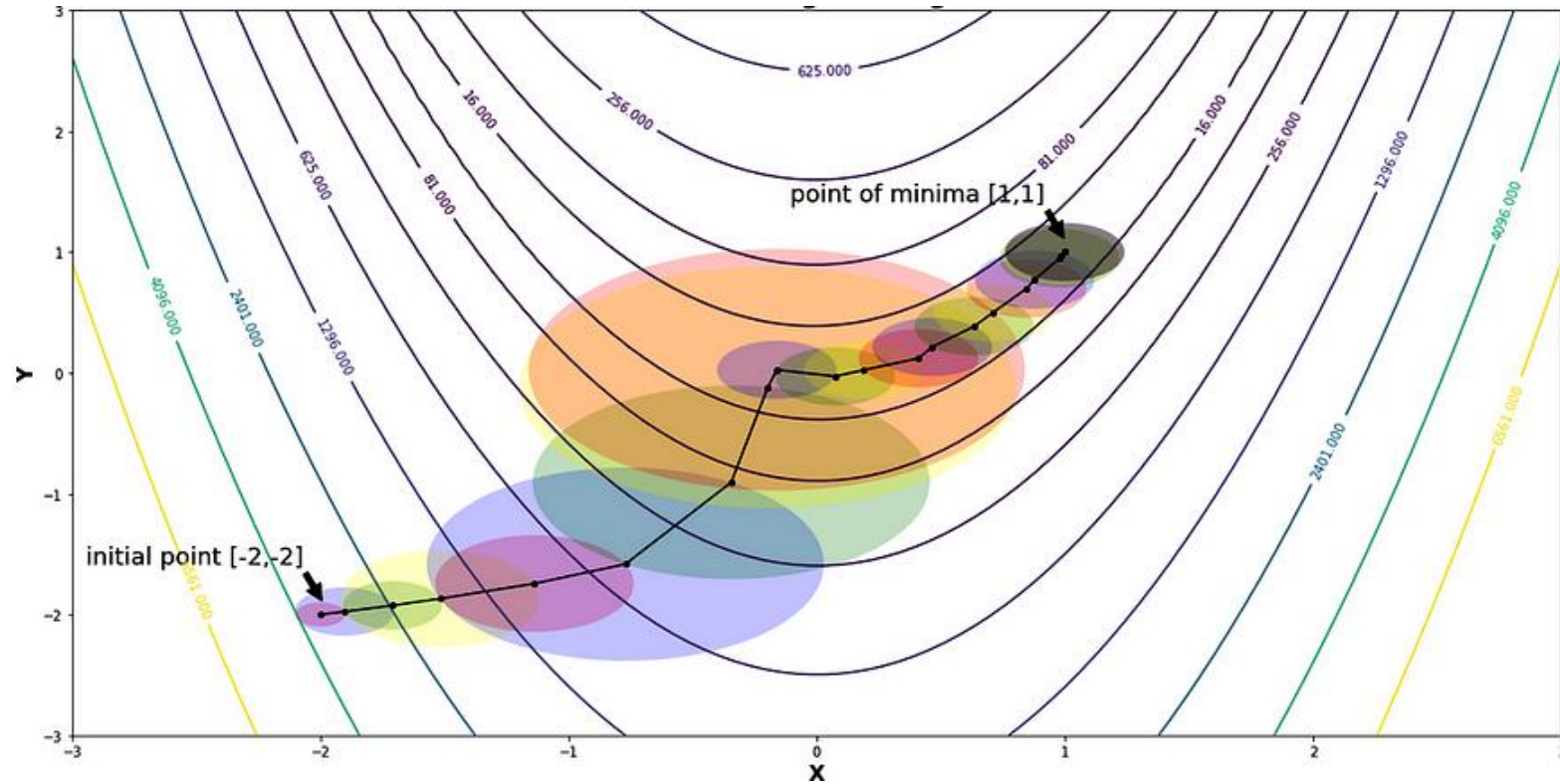
$$m_k(\mathbf{p}_k) = f_k + G_k \cdot \mathbf{p}_k + \frac{1}{2} \mathbf{p}_k^T \cdot H_k \cdot \mathbf{p}_k$$



Equivalent to
 $|\tau| \leq 1$

Level lines of m_k

Trust region method



Problem: Implement 2 steps of Cauchy point search for the Rosenbrock function $f(x_1, x_2) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$ starting at $(-2, -2)$ and with the trust regions being balls of radius 0.5 for both steps.

Summary:

- The Cauchy point is quick to calculate
- **It can be shown** that the Trust Region Method is globally convergent if its steps $\mathbf{p}_n = \mathbf{x}_n^* - \mathbf{x}_{n-1}^*$ attain sufficient reduction in the quadratic approximation
- **The Cauchy Point Algorithm serves as a benchmark for evaluating the performance of other methods.**

III. Descent direction?

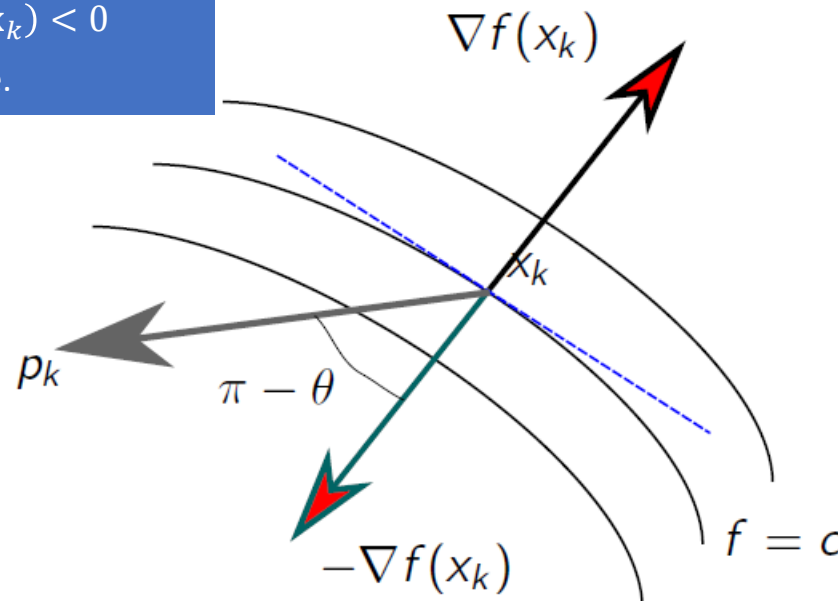
Definition. We say that p_k is a **descent direction** if $p_k^T \nabla f(\mathbf{x}_k) < 0$.
More generically (in line search methods) we consider

$$p_k = -B_k^{-1} \nabla f(\mathbf{x}_k) \quad \text{with } B_k \text{ positive definite.}$$

Observe that if B_k is positive definite, so is B_k^{-1} .
Therefore, if $p_k = -B_k^{-1} \cdot \nabla f(\mathbf{x}_k)$, then

$$\begin{aligned} p_k^T \cdot \nabla f(\mathbf{x}_k) &= -\left(B_k^{-1} \cdot \nabla f(\mathbf{x}_k)\right)^T \cdot \nabla f(\mathbf{x}_k) \\ &= -\nabla f(\mathbf{x}_k)^T \cdot (B_k^{-1})^T \cdot \nabla f(\mathbf{x}_k) < 0 \end{aligned}$$

Because $(B_k^{-1})^T$ is positive definite.



- $B_k = \text{Id}$ (**descent method**)
- $B_k = Hf(\mathbf{x}_k)$ (**Newton method**)
- $B_k \approx Hf(\mathbf{x}_k)$ (**quasi Newton method**)

Simplified notation:
 $H = \nabla^2 f$

(Rate of) Convergence?

Steepest descent method = descent along inverse gradient

The ideal case. Assume

$$f(x) = \frac{1}{2}x^T Qx - b^T x$$

where Q is symmetric and positive definite. The gradient is given by $\nabla f(x) = Qx - b$ and so the minimizer x^* is the (unique) solution of $Qx = b$. Algorithmically,

$$\min_{\alpha \in \mathbb{R}^+} f(x - \alpha_k \nabla f(x_k)) \quad \rightarrow \quad \hat{\alpha}_k = \frac{(\nabla f(x_k))^T \nabla f(x_k)}{(\nabla f(x_k))^T Q \nabla f(x_k)}$$

where notice that $\nabla f(x_k) = Qx_k - b$.

Steepest descent method

Definition. Accordingly we have that the steepest decent method with **exact line searches** writes as

$$x_{k+1} = x_k - \hat{\alpha}_k \nabla f(x_k)$$

To study the rate of convergence we introduce a weighted norm of a vector $x \in \mathbb{R}^n$ as follows

$$\|x\|_Q^2 = x^T Q x$$

Exercise: Show that $\|\cdot\|_Q$ is a norm.

Steepest descent method

$$f(x) = \frac{1}{2}x^T Qx - b^T x$$

Lemma. Assume above notation. We have

$$\frac{1}{2} \|x - x^*\|_Q^2 = f(x) - f(x^*).$$

Proof. The minimizer x^* satisfies $Qx^* = b$. Then

$$\begin{aligned} f(x^*) &= \frac{1}{2} \left((x^*)^T Qx^* - 2b^T x^* \right) = \frac{1}{2} \left((x^*)^T b - 2b^T x^* \right) = \\ &= -\frac{1}{2} b^T x^* = -\frac{1}{2} (x^*)^T Qx^*. \end{aligned}$$

where the last equality uses that $Q^T = Q$. Then

$$f(x) - f(x^*) = \frac{1}{2} \left(x^T Qx - 2b^T x + (x^*)^T Qx^* \right) = \frac{1}{2} \|x - x^*\|_Q^2$$

since $b^T x = (x^*)^T Qx$.

Steepest descent method

Theorem. When the steepest decent method with exact line searches ($\hat{\alpha}_k$) is applied to the strongly convex quadratic function above then

$$\|x_{k+1} - x^*\|_Q \leq \left(\frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1} \right) \cdot \|x_k - x^*\|_Q$$

where $0 < \lambda_1 \leq \dots \leq \lambda_n$ are the eigenvalues of Q .

Remark. The convergence of the steepest decent method under the best conditions, is **linear**.

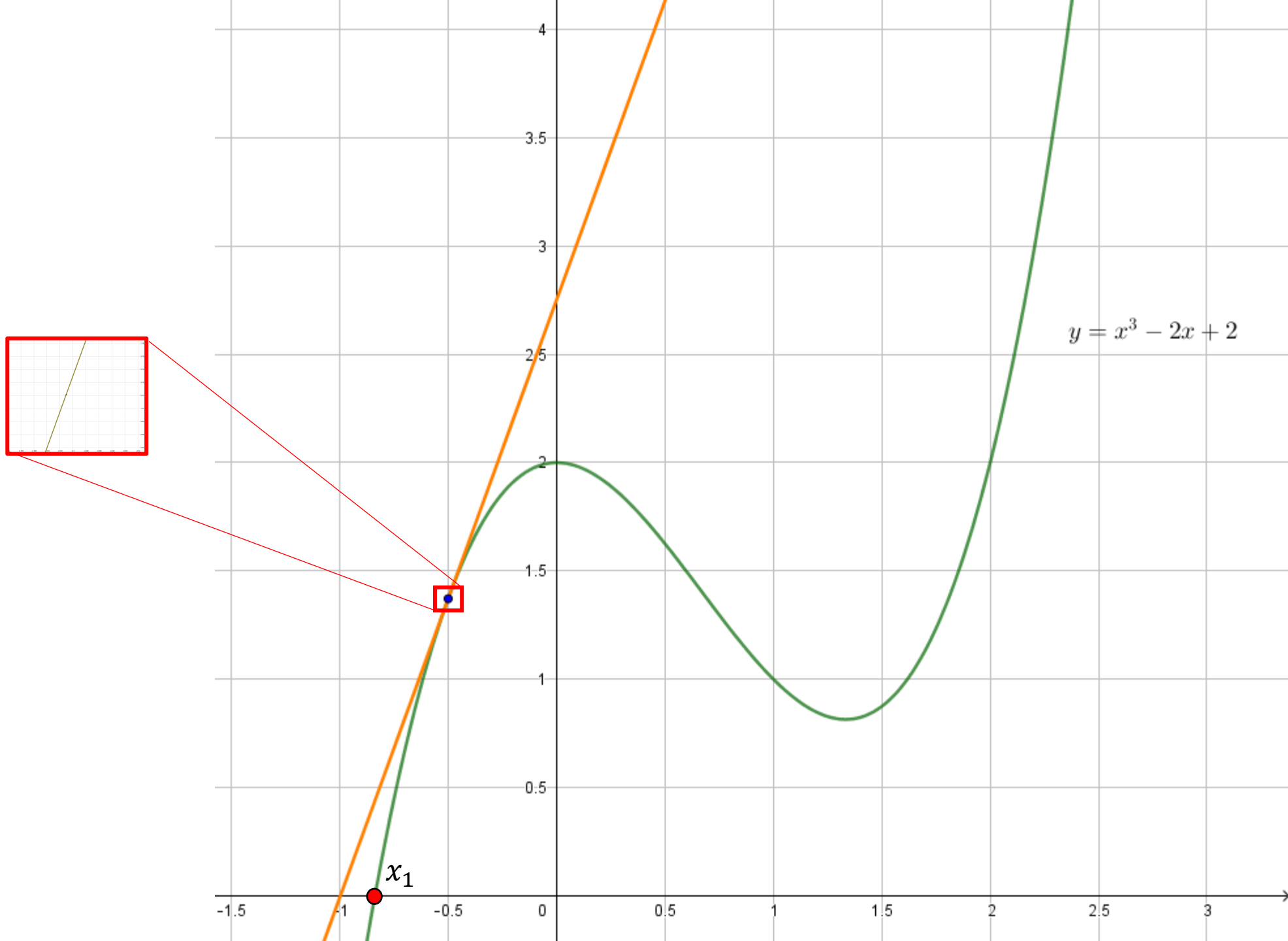
Newton's method

Definition. Let f twice differentiable. The **Newton's method** is the line search method defined by

$$p_k = - (Hf(x_k))^{-1} \nabla f(x_k).$$

Remark. Since $(Hf(x_k))^{-1}$ might not always be positive definite then Newton's method does not always define a **descent** method. However near the solutions (minimizers) the convergence is quadratic.





$$N_p(x) = x - \frac{p(x)}{p'(x)}$$

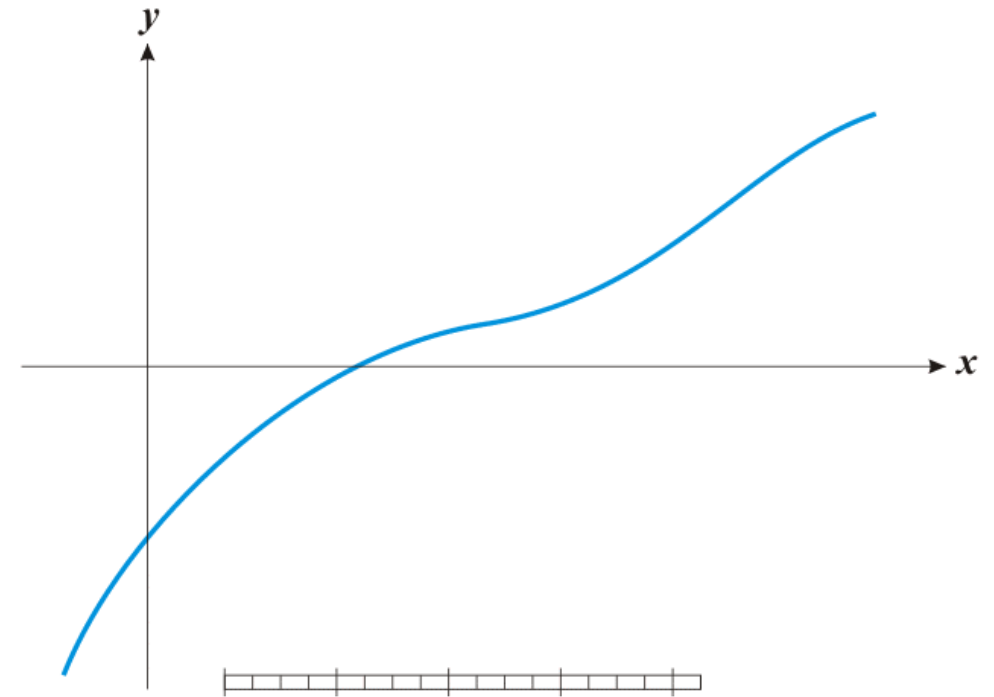
Newton map of $p(x)$

Theorem:

Let $\alpha \in \mathbb{R}$ be a root of the polynomial $p(x)$ (that is, $p(\alpha) = 0$).

Then for all x_1 sufficiently close to α , the iterates

$x_1, x_2 = N_p(x_1), x_3 = N_p(x_2) = N^{\circ 2}(x_1), \dots$ converges to α .



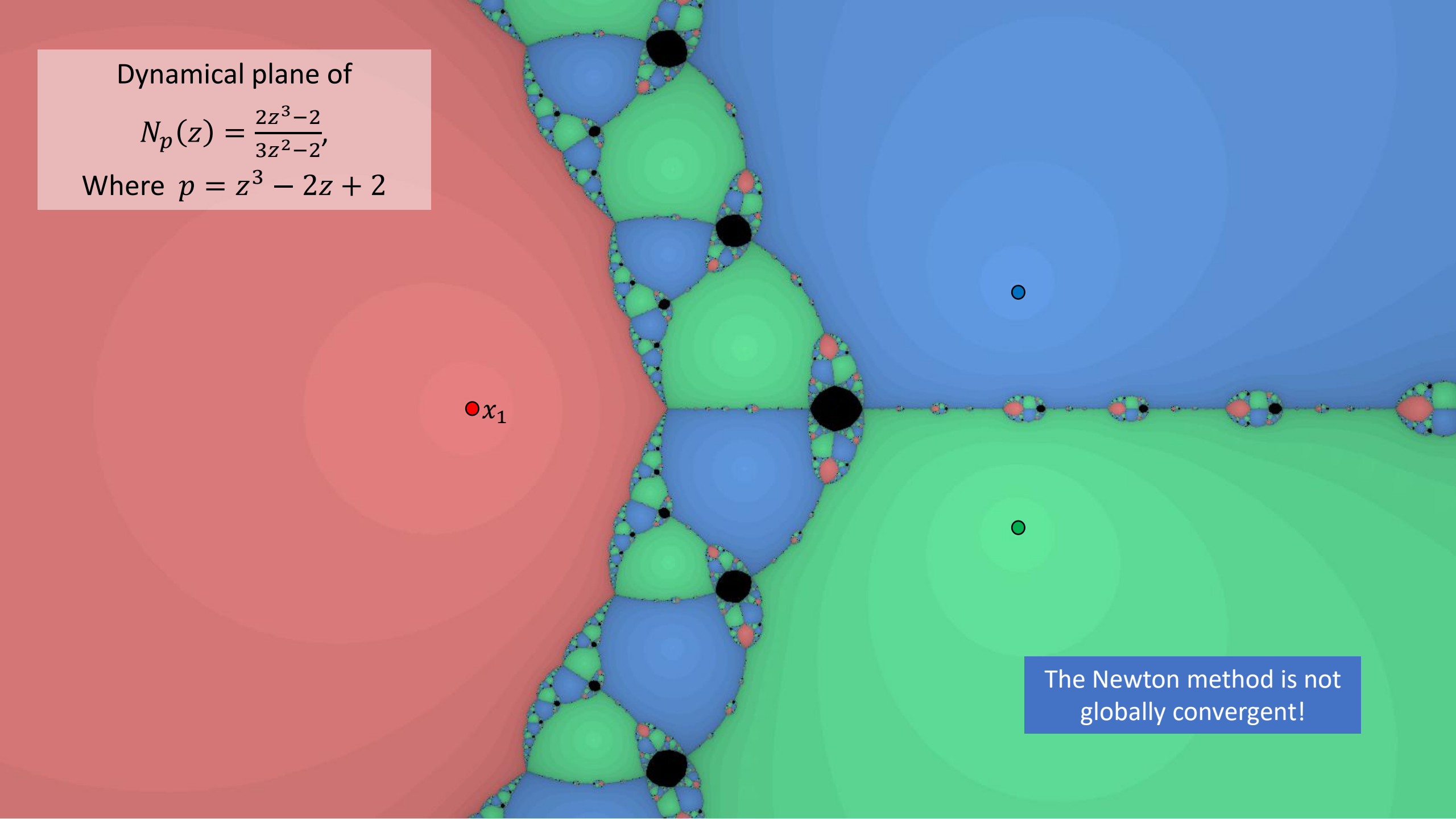
Dynamical plane of

$$N_p(z) = \frac{2z^3 - 2}{3z^2 - 2},$$

Where $p = z^3 - 2z + 2$

 x_1

The Newton method is not
globally convergent!



Dynamical plane of

$$N_p(z) = \frac{2z^3+1}{3z^2}, \text{ where } p = z^3 - 1$$

$$\left(-\frac{1}{2}, \frac{\sqrt{3}}{2}\right) \bullet$$

$$(1,0) \bullet$$

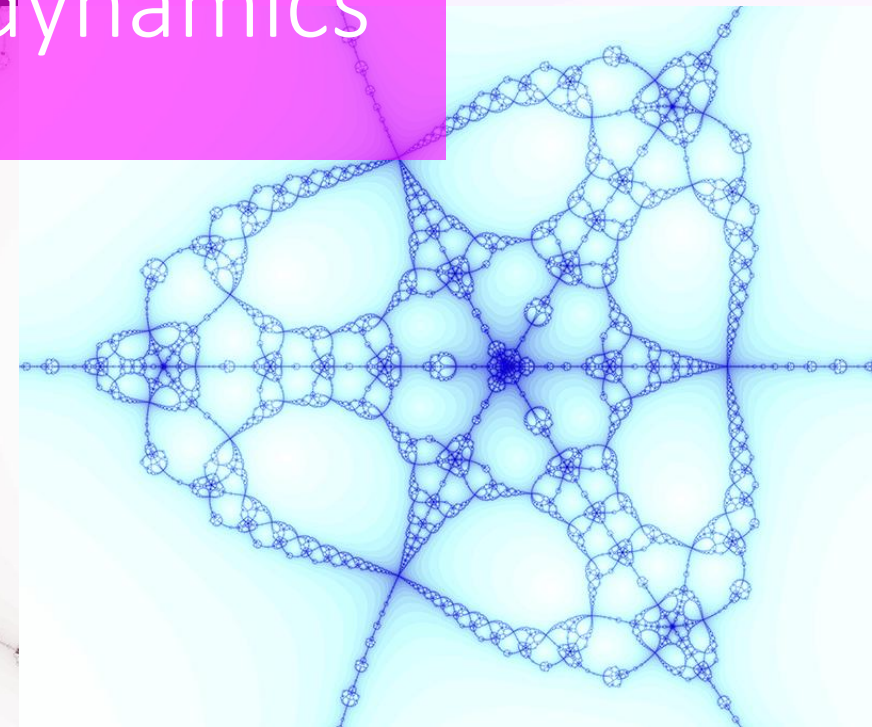
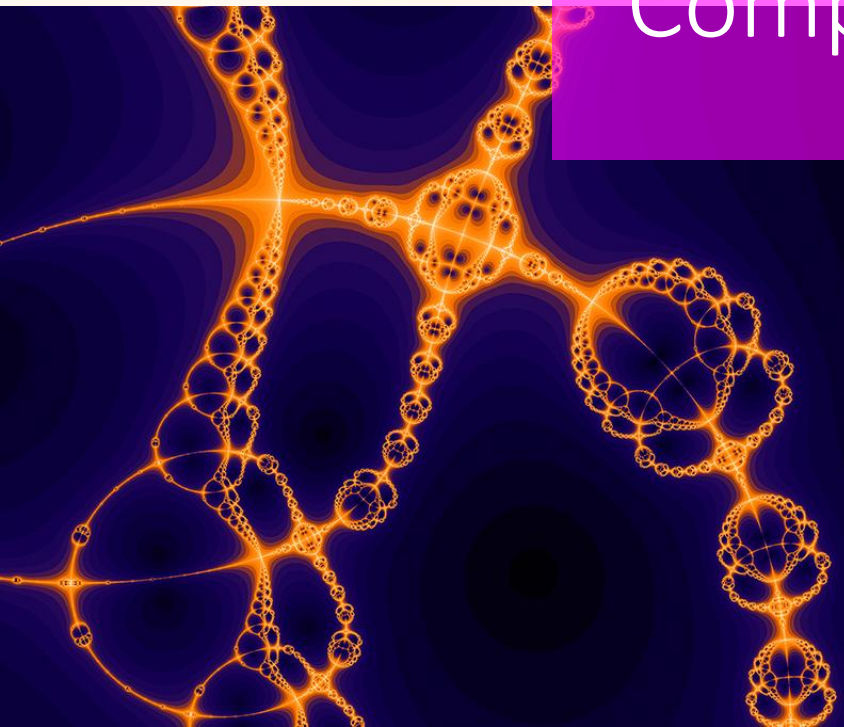
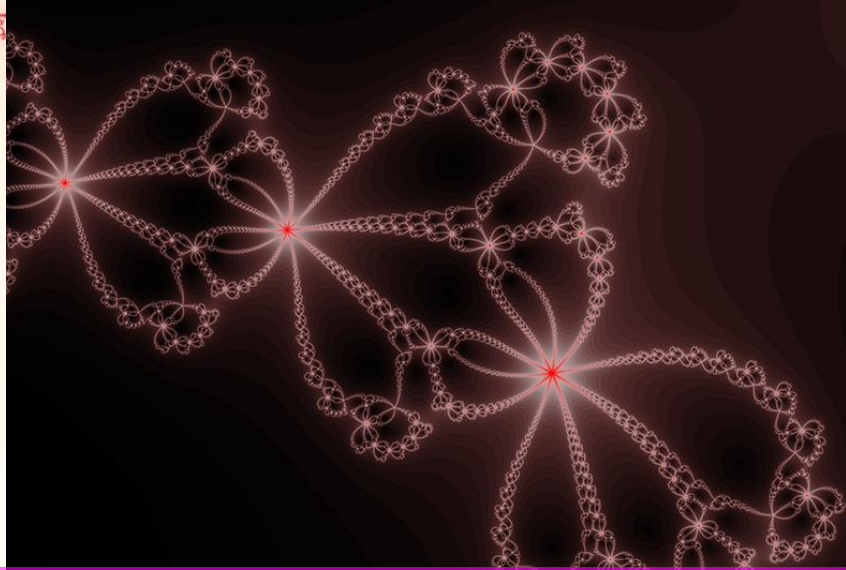
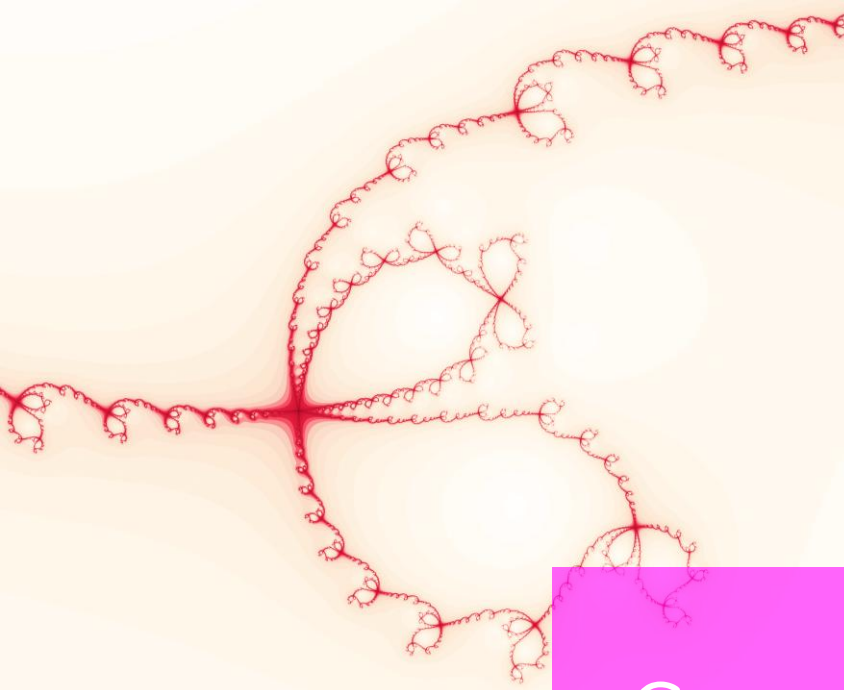
$$\left(-\frac{1}{2}, -\frac{\sqrt{3}}{2}\right) \bullet$$

$$\left(-\frac{1}{2}, \frac{\sqrt{3}}{2}\right) \bullet$$

$$(1,0) \bullet$$

$$\left(-\frac{1}{2}, -\frac{\sqrt{3}}{2}\right) \bullet$$

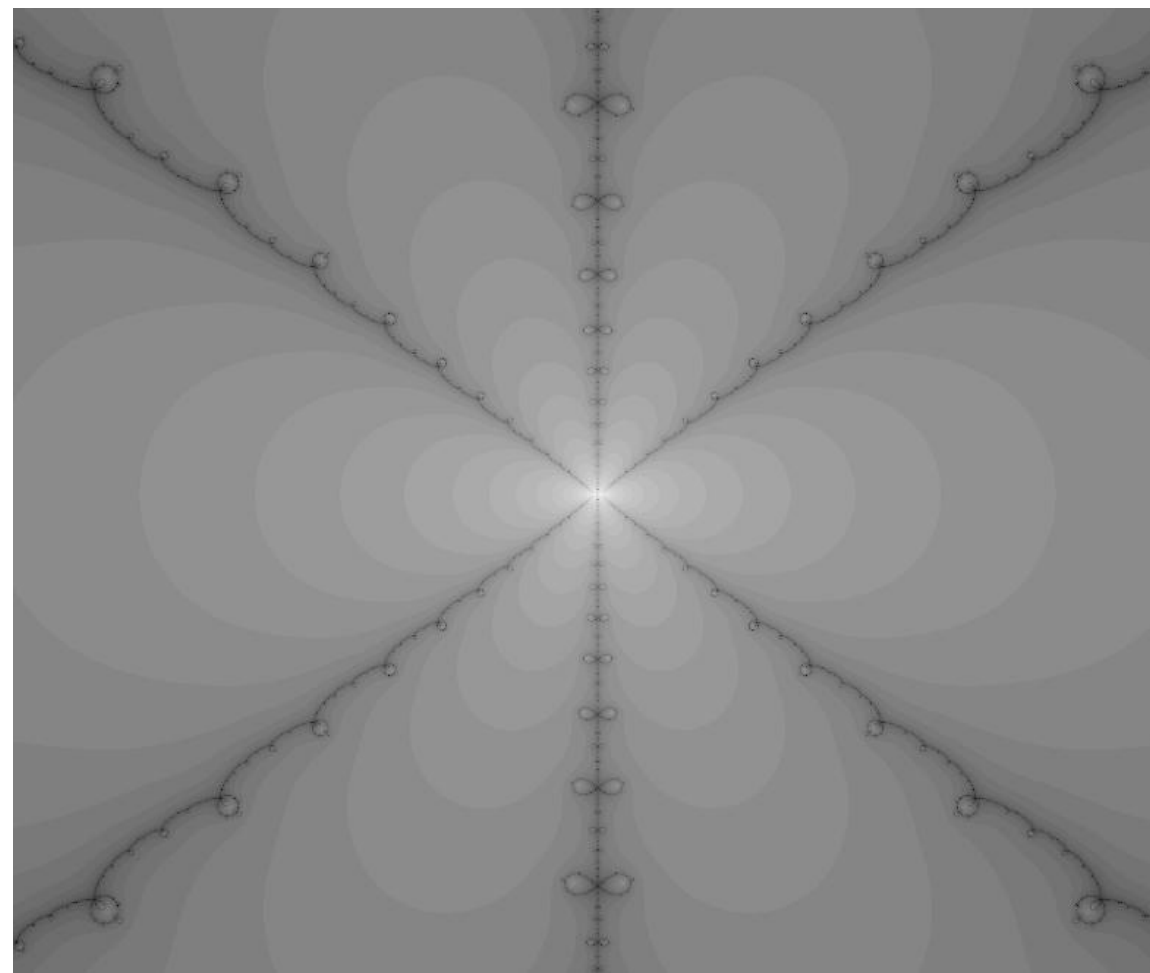
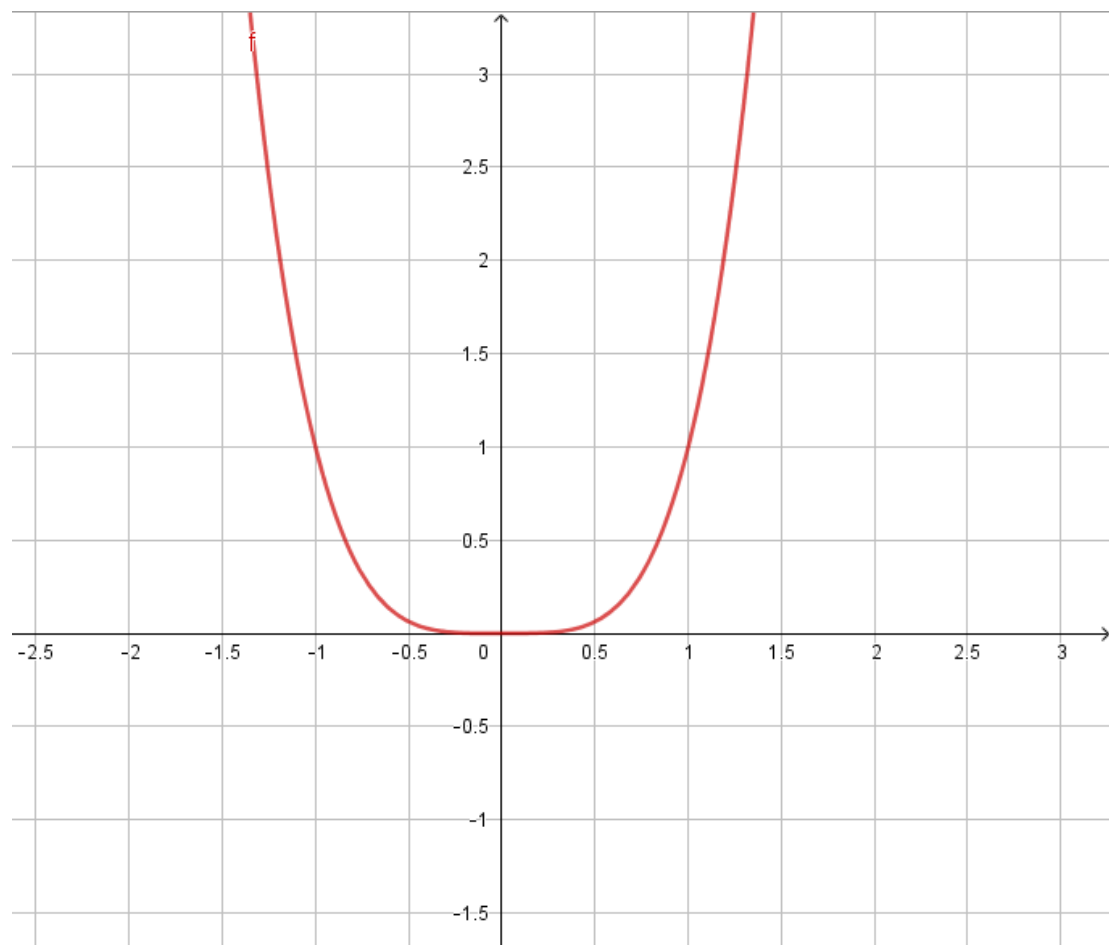
The Julia set = *a fractal*



Complex (holomorphic) dynamics

$$p(x) = (x + 0.02) \cdot (x - 0.01) \cdot (x - 0.02) \cdot (x + 0.01) =$$

$$= 4 \cdot 10^{-8} + 8.47033 \cdot 10^{-22} x - 0.0005 x^2 + 3.46945 \cdot 10^{-18} x^3 + x^4$$

 N_p


Theorem 1 (Main Theorem)

For every $d \geq 2$, there is a set $\mathcal{S}_d$ consisting of at most $1.11 d \log^2 d$ points in $\mathbb{C}$ with the property that for every polynomial $p \in \mathcal{P}_d$ and each of its roots, there is a point $s \in \mathcal{S}_d$ in the basin of the chosen root. For polynomials all of whose roots are real, there is an analogous set $\mathcal{S}$ with at most $1.3 d$ points.

$\mathcal{S}_d$

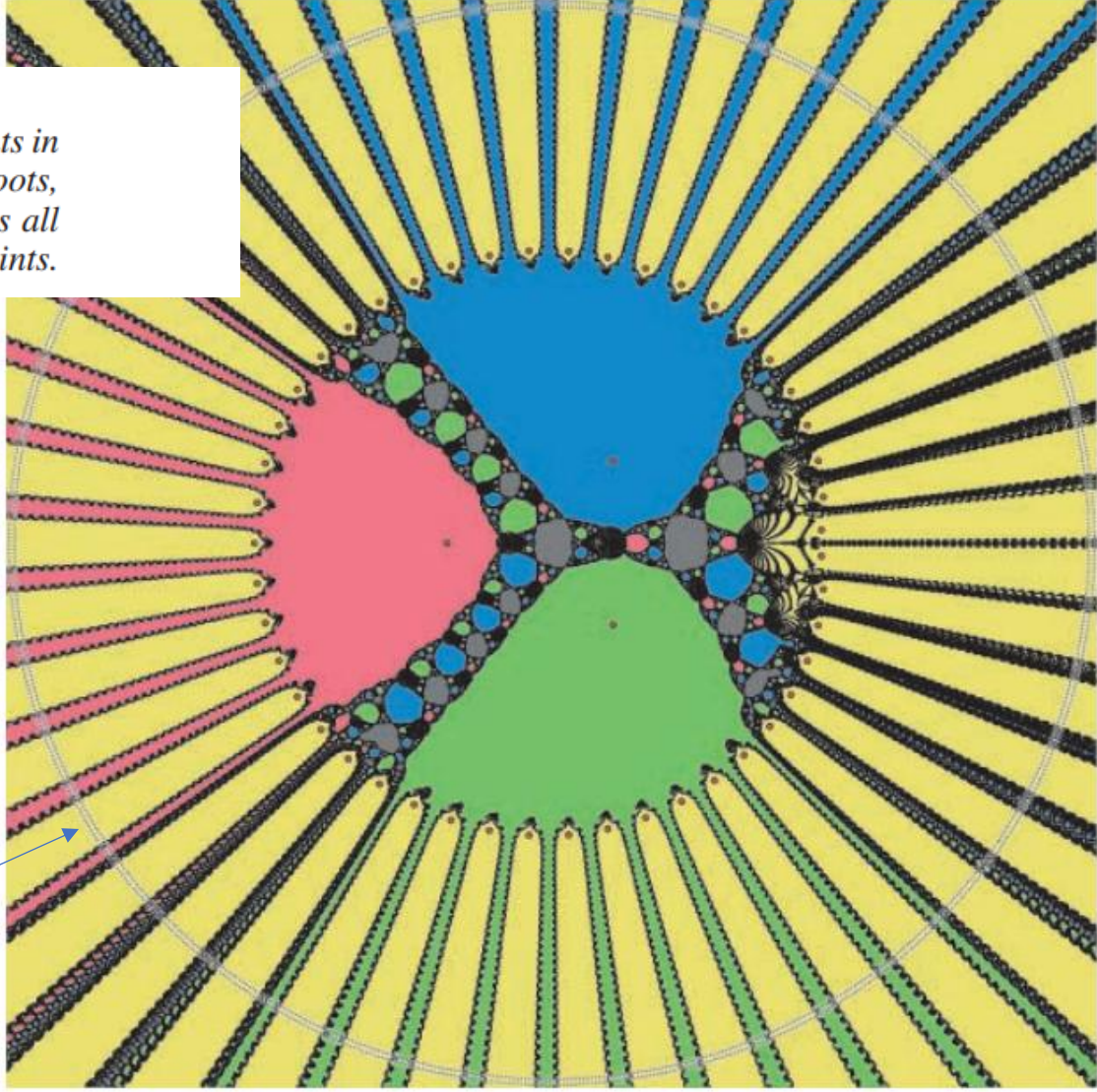


Fig. 2 A set of starting points as specified by our theorem for degree 50, indicated by small crosses distributed on two large circles. Also shown is the Julia set for the Newton map of the degree 50 polynomial $z^{50} + 8z^5 - \frac{80}{3}z^4 + 20z^3 - 2z + 1$ (black). There are 47 roots near the unit circle, and 3 roots well inside, all marked by red disks. As in Fig. 1, there is an attracting periodic orbit (basin in grey). A close-up is shown below

Newton's method

Definition. Let f twice differentiable. The **Newton's method** is the line search method defined by

$$p_k = - (Hf(x_k))^{-1} \nabla f(x_k) .$$

Remark. Since $(Hf(x_k))^{-1}$ might not always be positive definite then Newton's method does not always define a **descent** method. However near the solutions (minimizers) the convergence is quadratic.

Newton's method

Theorem. Assume f is regular (class $\mathcal{C}^3$ is enough) in a neighbourhood of a solution x^* (minimum of f) where the sufficient optimality conditions hold.
Consider the iteration

$$x_{k+1} = x_k + p_k$$

where p_k is the Newton direction expressed above. Then

- (a) $x_k \rightarrow x^*$, if x_0 is close enough to x^* .
- (b) The rate of convergence of $\{x_k\}_{k \geq 0}$ is quadratic.
- (c) $\|\nabla f(x_k)\| \rightarrow 0$ quadratically.

$\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^2$
 $\Rightarrow$ The number of significant digits
essentially doubles at each step

Newton's method

Proof:

Observe that $\nabla f(x^*) = 0$ (necessary condition for optimality). Thus,

$$\begin{aligned} x_k + p_k - x^* &= x_k - x^* - (Hf_k)^{-1} \nabla f(x_k) = \\ &= (Hf(x_k))^{-1} \cdot [Hf(x_k) \cdot (x_k - x^*) - \nabla f(x_k) + \nabla f(x^*)] \end{aligned}$$

Observe also that

$$\nabla f(x^*) - \nabla f(x_k) = \int_0^1 \frac{d}{dt} \nabla f(x_k - t(x_k - x^*)) dt = \int_0^1 Hf(x_k - t(x_k - x^*)) \cdot (x_k - x^*) dt$$

Therefore,

$$Hf(x_k) \cdot (x_k - x^*) - \nabla f(x_k) + \nabla f(x^*) = \int_0^1 \left(Hf(x_k) - Hf(x_k - t(x_k - x^*)) \right) \cdot (x_k - x^*) dt$$

Let $L > 0$ be the upper bound for the norm $\|Hf\|$ for all x in some small ball around x^* :

$$\|x - x^*\| \leq r.$$

This bound exists since f is smooth. Thus, for all x, y in this neighborhood,

$$\|Hf(x) - Hf(y)\| \leq L\|x - y\|$$

Hence,

$$\|Hf(x_k) \cdot (x_k - x^*) - \nabla f(x_k) + \nabla f(x^*)\| \leq L \cdot \|x_k - x^*\|^2 \int_0^1 t dt = \frac{L}{2} \cdot \|x_k - x^*\|^2$$

Newton's method

Proof (cont.): We go back to

$$\|x_k + p_k - x^*\| = \left\| (Hf(x_k))^{-1} \right\| \cdot \|Hf(x_k) \cdot (x_k - x^*) - \nabla f(x_k) + \nabla f(x^*)\|$$

We bounded the red. Using the C^3 regularity of f and the fact that $Hf(x^*)$ is not singular, we have

$$\left\| (Hf(x_k))^{-1} \right\| \leq 2 \left\| (Hf(x^*))^{-1} \right\| \quad \text{if} \quad \|x_k - x^*\| \leq r$$

(we can decrease r if necessary).

Finally,

$$\|x_{k+1} - x^*\| = \|x_k + p_k - x^*\| \leq L \cdot \left\| (Hf(x^*))^{-1} \right\| \cdot \|x_k - x^*\|^2 = C \cdot \|x_k - x^*\|^2$$

Choosing x_0 such that $\|x_0 - x^*\| < \frac{r}{C}$, we can use the inequality inductively to prove (a) and (b). Indeed,

$$\|x_k - x^*\| \leq C^{-1} \cdot (C \cdot \|x_0 - x^*\|)^{2^k}$$

Statement (c) can be proven using similar arguments.

Newton's method: discussion

For large systems, it might be computationally expensive to:

- Compute the inverse of the Hessian and do the matrix multiplication (provided we computed both the Hessian **and** the full gradient):

$$(Hf(x_k))^{-1} \cdot \nabla f(x_k)$$

- Compute the Hessian

- Compute the full gradient in the first place

Solve the linear system of equations instead:

$$(Hf(x_k))^{-1} \cdot \nabla f(x_k) = \mathbf{v} \Leftrightarrow Hf(x_k) \cdot \mathbf{v} = \nabla f(x_k)$$

Conjugate Gradient Method

Interpret second derivative as discrete first derivative

Quasi-Newton Method

Choose one component of the gradient vector at random

Stochastic Gradient Descent Method

Conjugate Gradient Method

The problem

Solving linear systems. Let A be a symmetric positive definite matrix ($n \times n$) and let $\mathbf{b} \in \mathbb{R}^n$. Then, solving the linear system

$$A \mathbf{x} = \mathbf{b} \quad (1)$$

is equivalent to solving the minimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^n} \varphi(\mathbf{x}) := \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}. \quad (2)$$

Definition. The **error** associated to the vector $\mathbf{x}$ as a solution of (1) will be represented by

$$r(\mathbf{x}) := A \mathbf{x} - \mathbf{b} = \nabla \varphi(\mathbf{x}). \quad (3)$$

We also use the notation $r_k := r(\mathbf{x}_k)$.

Conjugate Gradient Method

A conjugate set of vectors

Definition. Let A be a symmetric positive definite matrix ($n \times n$). A system of non-zero vectors $\{p_0, \dots, p_{n-1}\}$ is said to be **conjugate** with respect to A if

$$p_i^T A p_j = 0, \quad \text{for all } i \neq j. \quad (4)$$

Lemma. A conjugate system always exists and it is always linearly independent (base).

Proof. Exercise.

Conjugate Gradient Method

The conjugate direction method

Definition. Let $\mathbf{x}_0 \in \mathbb{R}^n$ and a conjugate system $\{p_0, \dots, p_{n-1}\}$ we generate the following finite sequence

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k p_k \quad (5)$$

where α_k is the one-dimensional minimizer of the quadratic function φ along the p_k -direction $(\mathbf{x}_k + \alpha p_k)$. In particular,

$$\alpha_k = -\frac{r_k^T p_k}{p_k^T A p_k}. \quad (\text{exercise})$$

Conjugate Gradient Method

The conjugate direction method

Theorem. For any $\mathbf{x}_0 \in \mathbb{R}^n$, the (finite) sequence $\{\mathbf{x}_k\}$ generated by the conjugate direction method (5) converges to the solution $\mathbf{x}^*$ of the linear system (1) in at most n steps.

Proof. Since the conjugate system is a base of $\mathbb{R}^n$ we have

$$\mathbf{x}^* - \mathbf{x}_0 = \beta_0 \mathbf{p}_0 + \cdots + \beta_{n-1} \mathbf{p}_{n-1},$$

for some (unique) choice of the parameters β_ℓ , $\ell = 0, \dots, n-1$. We claim that

$$\beta_\ell = \frac{\mathbf{p}_\ell^T A(\mathbf{x}^* - \mathbf{x}_0)}{\mathbf{p}_\ell^T A \mathbf{p}_\ell}.$$

To see the claim we pre-multiply the above expression by $\mathbf{p}_\ell^T A$ and use the conjugacy property.

Conjugate Gradient Method

The conjugate direction method

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \quad (5)$$

$$\mathbf{r}(\mathbf{x}) := \mathbf{A} \mathbf{x} - \mathbf{b} = \nabla \varphi(\mathbf{x}). \quad (3)$$

We also use the notation $r_k := r(\mathbf{x}_k)$.

Proof (continue).

On the other hand, using (5) we have that

$$\mathbf{x}_\ell = \mathbf{x}_0 + \alpha_0 \mathbf{p}_0 + \alpha_1 \mathbf{p}_1 + \cdots + \alpha_{\ell-1} \mathbf{p}_{\ell-1}.$$

Again by pre-multiplying the above expression by $\mathbf{p}_\ell^T \mathbf{A}$ and using the conjugacy property we have

$$\mathbf{p}_\ell^T \mathbf{A}(\mathbf{x}_\ell - \mathbf{x}_0) = 0.$$

Therefore

$$\mathbf{p}_\ell^T \mathbf{A}(\mathbf{x}^* - \mathbf{x}_0) = \mathbf{p}_\ell^T \mathbf{A}(\mathbf{x}^* - \mathbf{x}_\ell + \mathbf{x}_\ell - \mathbf{x}_0) = \mathbf{p}_\ell^T \mathbf{A}(\mathbf{x}^* - \mathbf{x}_\ell) = \mathbf{p}_\ell^T (\mathbf{b} - \mathbf{A}\mathbf{x}_\ell) = -\mathbf{p}_\ell^T \mathbf{r}_\ell$$

Now we have

$$\beta_\ell = \frac{\mathbf{p}_\ell^T \mathbf{A}(\mathbf{x}^* - \mathbf{x}_0)}{\mathbf{p}_\ell^T \mathbf{A} \mathbf{p}_\ell} = -\frac{\mathbf{r}_\ell^T \mathbf{p}_\ell}{\mathbf{p}_\ell^T \mathbf{A} \mathbf{p}_\ell} = \alpha_\ell$$

All together implies that $\mathbf{x}_\ell = \mathbf{x}^*$ for some $\ell = 0, \dots, n-1$.

Conjugate Gradient Method

The conjugate direction method

Definition. Given a system of vectors $\{v_1, \dots, v_k\}$ we denote by $\langle v_1, \dots, v_k \rangle$ the subspace generated by the vectors of the system.

Theorem. Let $\mathbf{x}_0 \in \mathbb{R}^n$ and consider the (finite) sequence $\{\mathbf{x}_k\}$ generated by the conjugate direction method (5). Then

$$r_k^T p_\ell = 0 \text{ for } \ell = 0, 1, \dots, k-1. \quad (6)$$

Moreover $\mathbf{x}_k$ is the minimizer of φ over the set

$$\{\mathbf{x} \mid \mathbf{x} = \mathbf{x}_0 + \langle p_0, \dots, p_{k-1} \rangle\}. \quad (7)$$

Conjugate Gradient Method

The conjugate direction method

$$r(\mathbf{x}) := A\mathbf{x} - \mathbf{b} = \nabla\varphi(\mathbf{x}). \quad (3)$$

We also use the notation $r_k := r(\mathbf{x}_k)$.

$$r_k^T p_\ell = 0 \text{ for } \ell = 0, 1, \dots, k-1. \quad (6)$$

Proof. We claim that a vector $\hat{\mathbf{x}}$ minimizes φ over the set (7) if and only if $r(\hat{\mathbf{x}})^T p_\ell = 0$ for all $\ell = 0, 1, \dots, k-1$. To see the claim we argue as follows. Let

$$h(\sigma) = \varphi\left(\mathbf{x}_0 + \sum_{\ell=0}^{k-1} \sigma_\ell p_\ell\right), \quad \sigma = (\sigma_0, \dots, \sigma_{k-1})^T.$$

Its unique minimizer, σ^* (h is strictly convex quadratic), satisfies

$$\frac{\partial h(\sigma^*)}{\partial \sigma_\ell} = 0, \quad \ell = 0, \dots, k-1.$$

So (chain rule)

$$\nabla\varphi\left(\mathbf{x}_0 + \sum_{\ell=0}^{k-1} \sigma_\ell^* p_\ell\right) p_\ell = r(\hat{\mathbf{x}})^T p_\ell = 0, \quad \ell = 0, \dots, k-1.$$

Now we should proof (6) (for all $\ell = 0, 1, \dots, k-1$). We use induction.

Conjugate Gradient Method

The conjugate direction method

Proof.

- The case $k = 1$ follows from the previous arguments. Since $\mathbf{x}_1 = \mathbf{x}_0 + \alpha_0 \mathbf{p}_0$ minimizes φ along $\mathbf{p}_0$ we have that $r_1^T \mathbf{p}_0 = 0$.
- Assume $r_{k-1}^T \mathbf{p}_\ell = 0$ for all $\ell = 0, \dots, k-2$. One can verify that $r_k = r_{k-1} + \alpha_{k-1} A \mathbf{p}_{k-1}$. So

$$\mathbf{p}_{k-1}^T r_k = \mathbf{p}_{k-1}^T r_{k-1} + \alpha_{k-1} \mathbf{p}_{k-1}^T A \mathbf{p}_{k-1} = 0$$

where the last equality follows from the definition of α_{k-1} . For the rest of conjugate directions $\mathbf{p}_\ell$, $\ell = 0, \dots, k-2$ we have

$$\mathbf{p}_\ell^T r_k = \mathbf{p}_\ell^T r_{k-1} + \alpha_{k-1} \mathbf{p}_\ell^T A \mathbf{p}_{k-1} = 0$$

because of the induction hypothesis and the fact that $\mathbf{p}_\ell$ is a conjugate vector with respect to A .

$$r(\mathbf{x}) := A \mathbf{x} - \mathbf{b} = \nabla \varphi(\mathbf{x}). \quad (3)$$

We also use the notation $r_k := r(\mathbf{x}_k)$.

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k \quad (5)$$

$$r_k^T \mathbf{p}_\ell = 0 \text{ for } \ell = 0, 1, \dots, k-1. \quad (6)$$

$$\alpha_k = -\frac{r_k^T \mathbf{p}_k}{\mathbf{p}_k^T A \mathbf{p}_k}.$$

Conjugate Gradient Method

The conjugate gradient method

Definition. The **conjugate gradient method** is a conjugate direction method such that in generating its set of conjugate vectors the **new** vector p_k can be computed using only the previous vector p_{k-1} . The algorithm gives for free the conjugacy property of p_k with respect to all previous $\{p_0, \dots, p_{k-1}\}$ vectors of the conjugate system. More concretely, p_0 is the steepest descent direction at the initial point x_0 and then we define

$$p_k = -r_k + \beta_k p_{k-1},$$

where β_k is determined such that $p_k^T A p_{k-1} = 0$.

Lemma. Under the above notation we have

$$\beta_k = \frac{r_k^T A p_{k-1}}{p_{k-1}^T A p_{k-1}}. \quad (\text{exercise})$$

Conjugate Gradient Method

Lemma. Suppose that the k -th iterate generated by the conjugate gradient method is not the solution point x^* . Then the following statements hold.

- (a) $r_k^T r_\ell = 0$ for $\ell = 0, \dots, k-1$.
- (b) $\langle r_0, \dots, r_k \rangle = \langle p_0, \dots, p_k \rangle = \langle r_0, Ar_0, \dots, A^k r_0 \rangle$.
- (c) $p_k^T A p_\ell = 0$ for $\ell = 0, \dots, k-1$.

Therefore, the (finite) sequence $\{x_k\}$ converges to x^* in at most n steps.

Proof (of (a)). Because the conjugate gradient method is a conjugate method we already have that

$$r_k^T p_\ell = 0 \quad \text{for } \ell = 0, \dots, k-1 \quad \text{and for all } k = 1, \dots, n-1.$$

But now since

$$p_\ell = -r_\ell + \beta_\ell p_{\ell-1}$$

we conclude that $r_\ell \in \langle p_\ell, p_{\ell-1} \rangle$. So,

$$r_k^T r_\ell = r_k^T (\hat{c}_1 p_\ell + \hat{c}_2 p_{\ell-1}) = 0 \quad \text{for all } \ell = 1, \dots, k-1.$$

By definition p_0 satisfies $r_k^T r_0 = -r_k^T p_0 = 0$ which finishes the proof.

Conjugate Gradient Method

The conjugate gradient method: Rate of convergence

Definition. We make the following choices of the parameters α_k and β_k as discussed above

$$\hat{\alpha}_k = -\frac{r_k^T p_k}{p_k^T A p_k} \quad \text{and} \quad \hat{\beta}_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$$

Conjugate Gradient Method

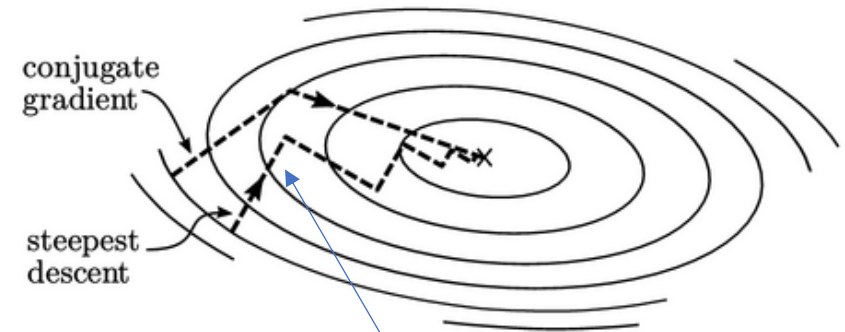
The conjugate gradient method: Rate of convergence

We might construct the following algorithm **CG: Conjugated gradient**.

Let $x_0 \in \mathbb{R}^n$. Set $r_0 \leftarrow Ax_0 - b$, $p_0 \leftarrow -r_0$, $k \leftarrow 0$. While $r_k \neq 0$.

$$\begin{aligned}\hat{\alpha}_k &\leftarrow -\frac{r_k^T p_k}{p_k^T A p_k} \\ x_{k+1} &\leftarrow x_k + \hat{\alpha}_k p_k \\ r_{k+1} &\leftarrow x_k + \hat{\alpha}_k p_k \\ \hat{\beta}_{k+1} &\leftarrow \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k} \\ p_{k+1} &\leftarrow -r_{k+1} + \hat{\beta}_{k+1} p_k \\ k &\leftarrow k + 1 \quad \text{end (while)}\end{aligned}\tag{8}$$

Theorem. If A has only $r \leq n$ distinct eigenvalues, then the CG method iteration will terminate at the solution in at most r iterations.



Problem: Show that in the steepest descent method with exact line search, the consecutive directions are orthogonal to each other.

Quasi-Newton Method

$\mathbb{R}^2$: **Secant method**

We don't want to compute $f'' \Rightarrow$

$$f''(x_k) \approx \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}}$$

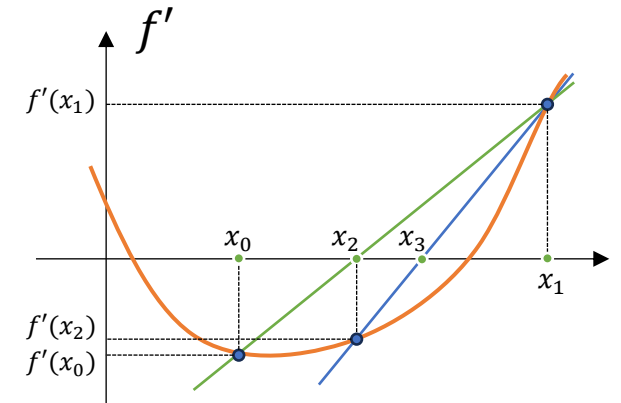
and the secant method is thus

given $x_0, x_1, k = 1$

repeat

- $x_{k+1} \leftarrow x_k - \frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} \cdot f'(x_k)$
- $k \leftarrow k + 1$

until the required precision is reached



Theorem (Convergence): For $f \in C^3$, if x^* is a strict local minimum of f , then for x_0, x_1 sufficiently close to x^* the sequence $(x_i)_{i=0}^{\infty}$ converges to x^* and there exists a constant $C \geq 1$ such that

$$|x_{k+1} - x^*| \leq C \cdot |x_k - x^*|^\phi,$$

where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.6$ is the Golden Ratio.

Super-linear rate of convergence:
- better than the steepest descent
- worse than Newton

Quasi-Newton Method

Now in $\mathbb{R}^n$

Idea: Substitute the inverse of the Hessian in the Newton method with a good approximation $\rightarrow$ use many gradients that we compute along the way (like in the secant method)

$$G_k := \nabla f(\mathbf{x}_k)$$

H_k := an **approximation** of a Hessian at step k (a symmetric, positive-definite matrix)

Suppose we have found $\mathbf{x}_{k+1}$ using the update $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k (H_k)^{-1} G_k$ (using the **line search methods** for α_k and the **conjugate gradient method** to find the direction $-(H_k)^{-1} G_k = \mathbf{v}_k \Leftrightarrow H_k \cdot \mathbf{v}_k = -G_k$)

Goal: Find $H_{k+1} \approx \nabla^2 f(\mathbf{x}_{k+1})$

Consider an auxiliary function

$$\begin{aligned} Q_{k+1}(\mathbf{x}) &:= G_{k+1}^T \cdot (\mathbf{x} - \mathbf{x}_{k+1}) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_{k+1})^T H_{k+1} (\mathbf{x} - \mathbf{x}_{k+1}) \\ \Rightarrow \nabla Q_{k+1}(\mathbf{x}) &= G_{k+1} + H_{k+1} \cdot (\mathbf{x} - \mathbf{x}_{k+1}) \end{aligned}$$

BFGS Algorithm (Broyden—Fletcher—Goldfarb—Shanno)



Quasi-Newton Method

Now in $\mathbb{R}^n$

$$Q_{k+1}(\mathbf{x}) = G_{k+1}^T \cdot (\mathbf{x} - \mathbf{x}_{k+1}) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_{k+1})^T H_{k+1} (\mathbf{x} - \mathbf{x}_{k+1})$$
$$\Rightarrow \nabla Q_{k+1}(\mathbf{x}) = G_{k+1} + H_{k+1} \cdot (\mathbf{x} - \mathbf{x}_{k+1})$$

We want to impose

Always

$$\begin{cases} \nabla Q_{k+1}(\mathbf{x}_{k+1}) = G_{k+1} \\ \nabla Q_{k+1}(\mathbf{x}_k) = G_k \end{cases} \Rightarrow H_{k+1} \cdot (\mathbf{x}_{k+1} - \mathbf{x}_k) = G_{k+1} - G_k \Rightarrow H_{k+1} \cdot \mathbf{s}_k = \mathbf{y}_k$$

Secant condition

Compare to $f''(x_k) \approx \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}}$

The matrix H_{k+1} is positive-definite if and only if $\mathbf{s}_k^T \mathbf{y}_k > 0$: can be *checked* (for certain classes of functions) or enforced (for instance, in the line search step). This condition is called the **curvature condition**.

Look for the solution in the form $H_{k+1} = H_k + \alpha \cdot \mathbf{u} \mathbf{u}^T + \beta \cdot \mathbf{v} \mathbf{v}^T$

$n \times n$ matrix

Choose: $\mathbf{u} = \mathbf{y}_k$, $\mathbf{v} = H_k \cdot \mathbf{s}_k$

Note that
 $w^T \mathbf{u} \mathbf{u}^T \mathbf{w} = (\mathbf{u}^T \mathbf{w})^2 > 0$
iff $w \neq 0$ and w is not orthogonal to $\mathbf{u}$

Hence, if H_k is positive-definite and symmetric (by induction), then H_{k+1} in the form above is also positive definite and symmetric.

Quasi-Newton Method

Now in $\mathbb{R}^n$

$$H_{k+1} \cdot \mathbf{s}_k = \mathbf{y}_k$$

Secant condition

Look for the solution in the form $H_{k+1} = H_k + \alpha \cdot \mathbf{u} \mathbf{u}^T + \beta \cdot \mathbf{v} \mathbf{v}^T$

Choose: $\mathbf{u} = \mathbf{y}_k$, $\mathbf{v} = H_k \cdot \mathbf{s}_k$

$$\mathbf{y}_k = H_{k+1} \mathbf{s}_k = H_k \mathbf{s}_k + \alpha \cdot \mathbf{y}_k \mathbf{y}_k^T \mathbf{s}_k + \beta \cdot H_k \mathbf{s}_k \cdot \mathbf{s}_k^T H_k^T \mathbf{s}_k$$

$$\Rightarrow \mathbf{y}_k (1 - \alpha \cdot \mathbf{y}_k^T \mathbf{s}_k) = H_k \mathbf{s}_k (1 + \beta \cdot \mathbf{s}_k^T H_k^T \mathbf{s}_k)$$

$$\Rightarrow \alpha = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k} \quad \beta = -\frac{1}{\mathbf{s}_k^T H_k^T \mathbf{s}_k}$$

> 0 because of the
curvature condition
 $\mathbf{y}_k^T \mathbf{s}_k > 0$

> 0 because H_k is positive
definite by inductive
assumption

Hence:

$$H_{k+1} = H_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{H_k \mathbf{s}_k \mathbf{s}_k^T H_k^T}{\mathbf{s}_k^T H_k^T \mathbf{s}_k}$$

Quasi-Newton Method

The BFGS Algorithm

Now in $\mathbb{R}^n$

given $\mathbf{x}_0, H_1 = id, k = 1$

repeat

1. Obtain a direction $\mathbf{v}_k$ by solving

$$H_k \cdot \mathbf{v}_k = -\nabla f(\mathbf{x}_k)$$

2. Perform line-search to find an acceptable step size α_k in the direction $\mathbf{v}_k$

3. Set

$$\mathbf{s}_k := \alpha_k \mathbf{v}_k$$

4. Update

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \mathbf{s}_k$$

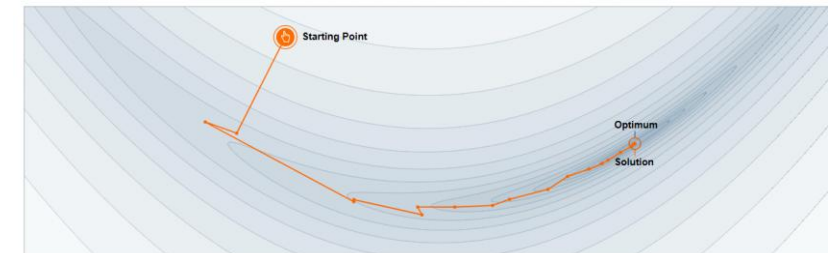
5. Set

$$\mathbf{y}_k := \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$$

6. Update

$$H_{k+1} \leftarrow H_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{H_k \mathbf{s}_k \mathbf{s}_k^T H_k^T}{\mathbf{s}_k^T H_k^T \mathbf{s}_k}$$

until the required precision is reached



The BFGS Algorithm has a super-linear rate of convergence:

- better than the steepest descent (linear)
- worse than Newton (quadratic)
- (but the BFGS algorithm is much faster!)